

تمرین سری دوم

هوش مصنوعی مقدماتی



MLP

محاسبات ریاضی و پروژه پیاده‌سازی شبکه‌های عصبی (MLP) برای دو مسئله کلاسیک یادگیری ماشین شامل طبقه‌بندی تصاویر ارقام (MNIST) و رگرسیون قیمت مسکن (Ames) می‌پردازد.

نام دانشجو:

حامد بهروزی مقدم

شماره دانشجویی:

۴۰۲۱۵۶۳۳

لینک کد سوال ۵: Q۵ Notebook Colab Google

لینک کد سوال ۶: Q۶ Notebook Colab Google



فهرست مطالب

۶	سوال ۱	۱
۶	۱.۱ صورت مسئله و تعریف شبکه	۶
۷	۲.۱ مشتق‌گیری برای تابع هزینه MSE	۷
۷	۱.۲.۱ مشتق نسبت به خروجی شبکه	۷
۸	۲.۲.۱ گرادیان‌های لایه خروجی	۸
۹	۳.۲.۱ گرادیان‌های لایه پنهان	۹
۱۰	۴.۲.۱ گرادیان‌های لایه اول	۱۰
۱۰	۵.۲.۱ جمع‌بندی حالت MSE	۱۰
۱۱	۳.۱ مشتق‌گیری برای تابع هزینه Cross-Entropy Binary	۱۱
۱۱	۱.۳.۱ مشتق نسبت به خروجی شبکه	۱۱
۱۲	۲.۳.۱ گرادیان‌های لایه خروجی	۱۲
۱۳	۳.۳.۱ گرادیان‌های لایه پنهان	۱۳
۱۳	۴.۳.۱ جمع‌بندی حالت BCE	۱۳
۱۴	۴.۱ مقایسه نهایی MSE و BCE	۱۴
۱۵	۵.۱ پاسخ نهایی	۱۵
۱۶	سوال ۲	۲
۱۶	۱.۲ صورت مسئله و تعریف شبکه	۱۶
۱۷	۲.۲ بخش الف: محاسبه بیشینه مشتق تابع سیگموئید	۱۷
۱۷	۱.۲.۲ محاسبه مشتق سیگموئید	۱۷
۱۹	۳.۲ بخش ب: بررسی سیگنال خطا و اثبات محو شدن گرادیان	۱۹
۲۰	۱.۳.۲ رابطه بازگشتی سیگنال خطا	۲۰
۲۰	۲.۳.۲ وابستگی خطا به حاصل ضرب مشتقات	۲۰
۲۱	۳.۳.۲ فرض نرمال بودن وزن‌ها	۲۱
۲۲	۴.۳.۲ اثبات نامساوی برای لایه اول	۲۲
۲۳	۴.۲ بخش ج: تحلیل اثر افزایش تعداد لایه‌ها بر سرعت یادگیری	۲۳
۲۴	۱.۴.۲ اثر روی به‌روزرسانی وزن‌ها	۲۴
۲۵	۵.۲ جمع‌بندی نهایی سوال ۲	۲۵



۲۶	سوال ۳	۳
۲۶	۱.۳ صورت مسئله و مفهوم قضیه تقریب عمومی	
۲۷	۲.۳ بخش الف: چرا به جای شبکه‌های بسیار عریض، از شبکه‌های عمیق استفاده می‌کنیم؟	
۲۷	۱.۲.۳ ۱. تفاوت بین توانایی وجودی و کارایی عملی	
۲۸	۲.۲.۳ ۲. بیان‌پذیری بهتر شبکه‌های عمیق	
۲۹	۳.۲.۳ ۳. کاهش تعداد پارامترها	
۳۰	۴.۲.۳ ۴. یادگیری ویژگی‌ها به صورت سلسله‌مراتبی	
۳۰	۵.۲.۳ ۵. سهولت یادگیری و تعمیم بهتر	
۳۱	۶.۲.۳ جمع‌بندی بخش الف	
۳۲	۳.۳ بخش ب: مثالی از تابعی که شبکه تک‌لایه برای نمایش آن به تعداد نمایی نوروں نیاز دارد	
۳۲	۱.۳.۳ مثال اول: تابع ضرب سلسله‌مراتبی	
۳۳	۲.۳.۳ نمایش تابع ضرب با شبکه عمیق	
۳۴	۳.۳.۳ مثال دوم: تابع ترکیبی سلسله‌مراتبی	
۳۵	۴.۳.۳ تفاوت رشد چندجمله‌ای و نمایی	
۳۶	۴.۳ تحلیل ارتباط با قضیه تقریب عمومی	
۳۷	۵.۳ جمع‌بندی نهایی سوال ۳	
۳۸	سوال ۴	۴
۳۸	۱.۴ بخش الف: الگوریتم Momentum	
۳۹	۱.۱.۴ تفسیر بردار سرعت	
۳۹	۲.۱.۴ اثر Momentum در راستای گرادیان‌های هم‌جهت	
۴۰	۳.۱.۴ اثر Momentum در راستای نوسان‌ها	
۴۰	۲.۴ بخش ب: الگوریتم RMSProp	
۴۱	۱.۲.۴ تفسیر جمله s_t	
۴۱	۲.۲.۴ تنظیم تطبیقی نرخ یادگیری	
۴۲	۳.۲.۴ اثر RMSProp در دره‌های باریک تابع هزینه	
۴۳	۳.۴ بخش ج: الگوریتم Adam	
۴۳	۱.۳.۴ اصلاح بایاس در Adam	
۴۴	۲.۳.۴ رابطه نهایی به‌روزرسانی در Adam	
۴۵	۳.۳.۴ نقش ابرپارامترهای Adam	
۴۵	۴.۳.۴ تغییر رفتار یادگیری با تنظیم پارامترها	
۴۶	۵.۳.۴ مقایسه Adam با Momentum و RMSProp	



۴۷	جمع بندی نهایی سوال ۴	۴.۴
۴۸	پیش بینی قیمت مسکن	۵
۴۸	۱.۵ (الف) بررسی اولیه داده	
۴۸	۱.۱.۵ بارگذاری داده و نمایش ۱۰ سطر اول	
۴۹	۲.۱.۵ گزارش تعداد نمونه ها و ویژگی ها	
۴۹	۳.۱.۵ بررسی ستون های دارای مقادیر گم شده (Missing Values)	
۵۰	۴.۱.۵ رسم هیستوگرام متغیر هدف و گزارش میانگین	
۵۱	۲.۵ (ب) پیش پردازش داده	
۵۱	۱.۲.۵ جایگزینی مقادیر گم شده	
۵۲	۲.۲.۵ تبدیل ویژگی های کیفی به عددی با Encoding One-Hot	
۵۳	۳.۲.۵ توضیح تعداد ویژگی ها پس از Encoding	
۵۳	۴.۲.۵ تقسیم داده ها به آموزش و آزمون	
۵۴	۵.۲.۵ نرمال سازی ویژگی ها با StandardScaler	
۵۵	۶.۲.۵ جمع بندی پارت ب	
۵۵	۳.۵ (پ) مدل پایه	
۵۵	۱.۳.۵ آموزش مدل Regression Linear	
۵۶	۲.۳.۵ گزارش مقدار RMSE مدل پایه	
۵۶	۴.۵ (ت) طراحی مدل MLP	
۵۶	۱.۴.۵ ساختار لایه های شبکه	
۵۸	۲.۴.۵ محاسبه پارامترهای قابل یادگیری	
۵۸	۳.۴.۵ دلیل استفاده از MSE در رگرسیون	
۵۸	۵.۵ (ث) آموزش مدل	
۵۸	۱.۵.۵ تنظیمات آموزش (Fit و Compile)	
۵۹	۲.۵.۵ تحلیل روند یادگیری	
۶۰	۳.۵.۵ ارزیابی نهایی و مقایسه با مدل خطی	
۶۱	۶.۵ (ج) بررسی ابرپارامترها	
۶۲	۱.۶.۵ (۱) بررسی توابع فعال سازی	
۶۴	۲.۶.۵ (۲) بررسی نرخ یادگیری	
۶۷	۳.۶.۵ (۳) بررسی معماری شبکه	
۷۰	۴.۶.۵ جمع بندی نهایی	
۷۰	۷.۵ (چ) تحلیل خطاهای مدل	



۷۰	استخراج نمونه‌های دارای بیشترین خطا	۱.۷.۵
۷۱	جدول نمونه‌های دارای بیشترین خطا	۲.۷.۵
۷۲	بررسی ویژگی‌های مؤثر بر خطا	۳.۷.۵
۷۳	تحلیل نهایی	۴.۷.۵
۷۴	۶ پیاده‌سازی شبکه عصبی چندلایه برای طبقه‌بندی MNIST	
۷۴	۱.۶ (الف) بارگذاری و آماده‌سازی داده‌ها	
۷۴	۱.۱.۶ بارگذاری مجموعه داده MNIST	
۷۶	۲.۱.۶ پیش‌پردازش تصاویر و برچسب‌ها	
۷۷	۲.۶ (ب) طراحی شبکه عصبی برای طبقه‌بندی	
۷۷	۱.۲.۶ طراحی ساختار کلی شبکه	
۷۸	۲.۲.۶ تابع فعال‌سازی Softmax	
۷۸	۳.۲.۶ تابع زیان Cross-Entropy در برابر MSE	
۷۸	۴.۲.۶ محاسبه تعداد پارامترها	
۷۹	۳.۶ (پ) آموزش مدل	
۷۹	۱.۳.۶ کامپایل مدل با Adam	
۸۰	۲.۳.۶ توضیح روند آموزش	
۸۰	۴.۶ (ت) ارزیابی مدل	
۸۰	۱.۴.۶ محاسبه دقت روی داده‌های آزمون	
۸۱	۲.۴.۶ ماتریس درهم‌ریختگی	
۸۲	۳.۴.۶ تحلیل خطاها	
۸۲	۵.۶ (ث) بررسی مقادیر اولیه وزن‌ها	
۸۲	۱.۵.۶ اثر مقداردهی اولیه تصادفی	
۸۳	۲.۵.۶ مقایسه با مقداردهی اولیه Xavier (Glorot)	
۸۴	۶.۶ (ج) بررسی اندازه Batch	
۸۴	۱.۶.۶ آزمایش با اندازه‌های مختلف Batch	
۸۵	۲.۶.۶ تحلیل تأثیر Batch Size	
۸۵	۷.۶ (چ) افزایش عمق شبکه	
۸۵	۱.۷.۶ مدل با لایه پنهان بیشتر	
۸۶	۲.۷.۶ تحلیل اینکه آیا مدل عمیق‌تر بهتر است یا نه	
۸۷	۸.۶ (ح) محدودیت‌های MLP در پردازش تصویر	
۸۷	۱.۸.۶ چرا Flatten لازم است؟	



۸۷	 مقایسه با شبکه‌های کانولوشنی CNN	۲.۸.۶
۸۷	 جمع‌بندی نهایی محدودیت‌ها	۳.۸.۶

۱ سوال ۱

۱.۱ صورت مسئله و تعریف شبکه

در این سوال یک شبکه پیشرو (Feedforward Neural Network) با یک لایه پنهان داریم که به صورت زیر تعریف شده است:

$$x \in \mathbb{R}^n$$

$$h = \sigma_h(W_1x + b_1)$$

$$\hat{y} = \sigma_o(W_2h + b_2)$$

که در آن:

- x ورودی شبکه است.
- W_1 و b_1 به ترتیب وزن ها و بایاس های لایه پنهان هستند.
- h خروجی لایه پنهان است.
- تابع فعال ساز لایه پنهان ReLU است:

$$\sigma_h(z) = \max(0, z)$$

- W_2 و b_2 به ترتیب وزن ها و بایاس های لایه خروجی هستند.
- تابع فعال ساز خروجی Sigmoid است:

$$\sigma_o(z) = \frac{1}{1 + e^{-z}}$$

نکته مهم

در این ساختار، اگر مسئله دسته بندی دودویی باشد، معمولاً تابع هزینه Binary Cross-Entropy انتخاب طبیعی تر و مناسب تر از MSE است؛ زیرا با خروجی سیگموئید سازگارتر بوده و گرادیان های بهتری برای آموزش ایجاد می کند.



برای سادگی، ابتدا تعریف می‌کنیم:

$$z_1 = W_1x + b_1 \quad \Rightarrow \quad h = \sigma_h(z_1)$$

$$z_2 = W_2h + b_2 \quad \Rightarrow \quad \hat{y} = \sigma_o(z_2)$$

هدف مسئله، استخراج مشتقات زیر نسبت به پارامترهاست:

$$\frac{\partial L}{\partial W_2}, \quad \frac{\partial L}{\partial b_2}, \quad \frac{\partial L}{\partial W_1}, \quad \frac{\partial L}{\partial b_1}$$

—

۲.۱ مشتق‌گیری برای تابع هزینه MSE

تابع هزینه میانگین مربعات خطا برای یک نمونه به صورت زیر است:

$$L_{MSE} = \frac{1}{2}(\hat{y} - y)^2$$

نکته کلیدی

عامل $\frac{1}{2}$ فقط برای ساده‌سازی مشتق‌گیری اضافه شده است، چون مشتق $(\cdot)^2$ ضریب 2 ایجاد می‌کند و این ضریب با $\frac{1}{2}$ خنثی می‌شود.

۱.۲.۱ مشتق نسبت به خروجی شبکه

ابتدا داریم:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

از طرفی چون:

$$\hat{y} = \sigma_o(z_2)$$

و برای سیگموئید:

$$\sigma'_o(z_2) = \hat{y}(1 - \hat{y})$$

پس:

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

رابطه مهم

$$\delta_2 \equiv \frac{\partial L}{\partial z_2} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

این کمیت همان خطای لایه خروجی است.

۲.۲.۱ گرادیان‌های لایه خروجی

چون:

$$z_2 = W_2 h + b_2$$

داریم:

$$\frac{\partial z_2}{\partial W_2} = h, \quad \frac{\partial z_2}{\partial b_2} = 1$$

پس:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial W_2} = \delta_2 h$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial b_2} = \delta_2$$

بنابراین:

$$\frac{\partial L_{MSE}}{\partial W_2} = (\hat{y} - y)\hat{y}(1 - \hat{y}) h$$

$$\frac{\partial L_{MSE}}{\partial b_2} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

۳.۲.۱ گرادیان‌های لایه پنهان

اکنون باید خطا را به لایه پنهان منتقل کنیم. چون:

$$z_2 = W_2 h + b_2$$

داریم:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial h} = \delta_2 W_2$$

از طرف دیگر:

$$h = \sigma_h(z_1) = \text{ReLU}(z_1)$$

و مشتق ReLU برابر است با:

$$\sigma'_h(z_1) = \begin{cases} 1 & z_1 > 0 \\ 0 & z_1 < 0 \end{cases}$$

(در نقطه $z_1 = 0$ معمولاً یکی از مقادیر 0 یا به صورت قراردادی یک زیرمشتق انتخاب می‌شود).
پس:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \sigma'_h(z_1) = (\delta_2 W_2) \odot \sigma'_h(z_1)$$

که $\odot$ ضرب عضو به عضو است.

اگر تعریف کنیم:

$$\delta_1 \equiv \frac{\partial L}{\partial z_1}$$

آنگاه:

$$\delta_1 = (\delta_2 W_2) \odot \sigma'_h(z_1)$$

نکته مهم

در لایه‌های پنهان، گرادیان فقط از وزن‌های لایه بعدی عبور نمی‌کند؛ بلکه باید در مشتق تابع فعال‌ساز همان لایه نیز ضرب شود. برای ReLU این مشتق یا 1 است یا 0، بنابراین نرون‌های خاموش، گرادیان را عبور نمی‌دهند.



۴.۲.۱ گرادیان‌های لایه اول

از آنجا که:

$$z_1 = W_1 x + b_1$$

داریم:

$$\frac{\partial z_1}{\partial W_1} = x, \quad \frac{\partial z_1}{\partial b_1} = 1$$

بنابراین:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial W_1} = \delta_1 x$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b_1} = \delta_1$$

پس در نهایت:

$$\frac{\partial L_{MSE}}{\partial W_1} = [((\hat{y} - y)\hat{y}(1 - \hat{y})) W_2] \odot \sigma'_h(z_1) x$$

$$\frac{\partial L_{MSE}}{\partial b_1} = [((\hat{y} - y)\hat{y}(1 - \hat{y})) W_2] \odot \sigma'_h(z_1)$$

۵.۲.۱ جمع‌بندی حالت MSE

اگر به صورت خلاصه بنویسیم، برای یک نمونه:

$$\delta_2 = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

$$\frac{\partial L}{\partial W_2} = \delta_2 h \quad \frac{\partial L}{\partial b_2} = \delta_2$$

$$\delta_1 = (\delta_2 W_2) \odot \sigma'_h(z_1)$$



$$\frac{\partial L}{\partial W_1} = \delta_1 x \quad \frac{\partial L}{\partial b_1} = \delta_1$$

فرم نهایی MSE

$$\begin{aligned} \frac{\partial L_{MSE}}{\partial W_2} &= (\hat{y} - y) \hat{y} (1 - \hat{y}) h \\ \frac{\partial L_{MSE}}{\partial b_2} &= (\hat{y} - y) \hat{y} (1 - \hat{y}) \\ \frac{\partial L_{MSE}}{\partial W_1} &= [((\hat{y} - y) \hat{y} (1 - \hat{y})) W_2] \odot \sigma'_h(z_1) x \\ \frac{\partial L_{MSE}}{\partial b_1} &= [((\hat{y} - y) \hat{y} (1 - \hat{y})) W_2] \odot \sigma'_h(z_1) \end{aligned}$$

۳.۱ مشتق‌گیری برای تابع هزینه Cross-Entropy Binary

تابع هزینه کراس انتروپی دودویی به صورت زیر است:

$$L_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

مزیت مهم BCE

وقتی خروجی شبکه $\sigma_o = \text{Sigmoid}$ باشد و مسئله دودویی باشد، ترکیب Sigmoid و BCE منجر به فرم بسیار ساده و پایدار گرادیان می‌شود.

۱.۳.۱ مشتق نسبت به خروجی شبکه

ابتدا مشتق نسبت به $\hat{y}$ را حساب می‌کنیم:

$$\frac{\partial L}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)$$

اکنون مشتق سیگموئید را داریم:

$$\frac{\partial \hat{y}}{\partial z_2} = \hat{y}(1 - \hat{y})$$

پس:



$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2}$$

$$= - \left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right) \hat{y}(1-\hat{y})$$

با ساده‌سازی:

$$\frac{\partial L}{\partial z_2} = \hat{y} - y$$

نتیجه بسیار مهم

$$\boxed{\frac{\partial L_{BCE}}{\partial z_2} = \hat{y} - y}$$

این یکی از مهم‌ترین نتایج در یادگیری عمیق است؛ یعنی برای Sigmoid + BCE، مشتق لایه خروجی بسیار ساده می‌شود.

۲.۳.۱ گرادینت‌های لایه خروجی

چون:

$$z_2 = W_2 h + b_2$$

پس:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial W_2} = (\hat{y} - y) h$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} = \hat{y} - y$$

بنابراین:

$$\boxed{\frac{\partial L_{BCE}}{\partial W_2} = (\hat{y} - y) h}$$

$$\boxed{\frac{\partial L_{BCE}}{\partial b_2} = \hat{y} - y}$$



۳.۳.۱ گرادیان‌های لایه پنهان

اکنون:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial h} = (\hat{y} - y)W_2$$

و با توجه به ReLU:

$$\frac{\partial L}{\partial z_1} = ((\hat{y} - y)W_2) \odot \sigma'_h(z_1)$$

بنابراین:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial W_1} = [((\hat{y} - y)W_2) \odot \sigma'_h(z_1)] x$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} = ((\hat{y} - y)W_2) \odot \sigma'_h(z_1)$$

پس در نهایت:

$$\frac{\partial L_{BCE}}{\partial W_1} = [((\hat{y} - y)W_2) \odot \sigma'_h(z_1)] x$$

$$\frac{\partial L_{BCE}}{\partial b_1} = ((\hat{y} - y)W_2) \odot \sigma'_h(z_1)$$

—

۴.۳.۱ جمع‌بندی حالت BCE

برای تابع هزینه Binary Cross-Entropy:

$$\delta_2 = \hat{y} - y$$

$$\frac{\partial L}{\partial W_2} = \delta_2 h \quad \frac{\partial L}{\partial b_2} = \delta_2$$

$$\delta_1 = (\delta_2 W_2) \odot \sigma'_h(z_1)$$

$$\frac{\partial L}{\partial W_1} = \delta_1 x \quad \frac{\partial L}{\partial b_1} = \delta_1$$



فرم نهایی BCE

$$\begin{aligned}\frac{\partial L_{BCE}}{\partial W_2} &= (\hat{y} - y) h \\ \frac{\partial L_{BCE}}{\partial b_2} &= \hat{y} - y \\ \frac{\partial L_{BCE}}{\partial W_1} &= [((\hat{y} - y)W_2) \odot \sigma'_h(z_1)] x \\ \frac{\partial L_{BCE}}{\partial b_1} &= ((\hat{y} - y)W_2) \odot \sigma'_h(z_1)\end{aligned}$$

۴.۱ مقایسه نهایی MSE و BCE

مقایسه مهم

- در MSE، گرادیان لایه خروجی شامل ضریب $\hat{y}(1 - \hat{y})$ است.
- در BCE، این ضریب حذف می‌شود و نتیجه نهایی بسیار ساده‌تر است.
- بنابراین در مسائل دسته‌بندی دودویی، BCE معمولاً انتخاب بهتر و پایدارتر است.

اگر بخواهیم از نظر عددی مقایسه کنیم:

$$\text{MSE: } \frac{\partial L}{\partial z_2} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

$$\text{BCE: } \frac{\partial L}{\partial z_2} = \hat{y} - y$$

در نتیجه، زمانی که $\hat{y}$ به 0 یا 1 نزدیک می‌شود، مشتق MSE ممکن است بسیار کوچک شود و یادگیری را کند کند؛ اما BCE این مشکل را تا حد زیادی کاهش می‌دهد.

۵.۱ پاسخ نهایی

پاسخ نهایی فشرده

برای این شبکه داریم:

$$z_1 = W_1 x + b_1, \quad h = \text{ReLU}(z_1), \quad z_2 = W_2 h + b_2, \quad \hat{y} = \sigma(z_2)$$

برای MSE:

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y) \hat{y} (1 - \hat{y}) h$$

$$\frac{\partial L}{\partial b_2} = (\hat{y} - y) \hat{y} (1 - \hat{y})$$

$$\frac{\partial L}{\partial W_1} = [((\hat{y} - y) \hat{y} (1 - \hat{y})) W_2] \odot \sigma'_h(z_1) x$$

$$\frac{\partial L}{\partial b_1} = [((\hat{y} - y) \hat{y} (1 - \hat{y})) W_2] \odot \sigma'_h(z_1)$$

برای BCE:

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y) h$$

$$\frac{\partial L}{\partial b_2} = \hat{y} - y$$

$$\frac{\partial L}{\partial W_1} = [((\hat{y} - y) W_2) \odot \sigma'_h(z_1)] x$$

$$\frac{\partial L}{\partial b_1} = ((\hat{y} - y) W_2) \odot \sigma'_h(z_1)$$



سوال ۲

۱.۲ صورت مسئله و تعریف شبکه

در این سوال یک شبکه عصبی عمیق با L لایه در نظر گرفته شده است که در تمام لایه‌های آن از تابع فعال‌ساز سیگموید استفاده می‌شود:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

هدف سوال بررسی این موضوع است که چرا در شبکه‌های عمیق، استفاده از سیگموید می‌تواند باعث پدیده‌ای به نام محو شدن گرادیان یا Vanishing Gradient شود.

ایده اصلی سوال

در شبکه‌های عمیق، گرادیان برای رسیدن از لایه‌های انتهایی به لایه‌های ابتدایی باید چندین بار در مشتق تابع فعال‌ساز و وزن‌ها ضرب شود. اگر این ضرایب کوچک‌تر از 1 باشند، مقدار گرادیان به صورت نمایی کوچک می‌شود.

برای لایه k -ام شبکه می‌نویسیم:

$$z^{(k)} = W^{(k)}a^{(k-1)} + b^{(k)}$$

$$a^{(k)} = \sigma(z^{(k)})$$

که در آن:

- $z^{(k)}$ ورودی خالص لایه k -ام است.
- $a^{(k)}$ خروجی یا فعال‌سازی لایه k -ام است.
- $W^{(k)}$ ماتریس وزن‌های لایه k -ام است.
- $b^{(k)}$ بردار بایاس لایه k -ام است.
- $\sigma(\cdot)$ تابع سیگموید است.

همچنین سیگنال خطا در لایه k -ام را به صورت زیر تعریف می‌کنیم:

$$\delta^{(k)} = \frac{\partial L}{\partial z^{(k)}}$$

که نشان دهنده حساسیت تابع هزینه نسبت به ورودی خالص لایه k -ام است.

۲.۲ بخش الف: محاسبه بیشینه مشتق تابع سیگموید

تابع سیگموید برابر است با:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

برای بررسی محو شدن گرادیان، ابتدا باید مشتق این تابع را محاسبه کنیم.

۱.۲.۲ محاسبه مشتق سیگموید

داریم:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

مشتق آن نسبت به z برابر است با:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

اما یک فرم بسیار مهم تر و کاربردی تر برای مشتق سیگموید وجود دارد. چون:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

می توان نشان داد:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

فرمول مهم مشتق سیگموید

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

این رابطه در تحلیل شبکه های عصبی بسیار مهم است، زیرا نشان می دهد مشتق سیگموید فقط به خروجی خود سیگموید وابسته است.



اکنون برای پیدا کردن بیشینه مشتق، قرار می دهیم:

$$s = \sigma(z)$$

از آنجا که خروجی سیگموید همیشه بین صفر و یک است، داریم:

$$0 < s < 1$$

پس مشتق سیگموید به صورت زیر نوشته می شود:

$$\sigma'(z) = s(1 - s)$$

اکنون تابع زیر را در بازه $(0, 1)$ بیشینه می کنیم:

$$f(s) = s(1 - s) = s - s^2$$

مشتق این تابع نسبت به s برابر است با:

$$f'(s) = 1 - 2s$$

برای یافتن نقطه بحرانی:

$$f'(s) = 0$$

پس:

$$1 - 2s = 0$$

$$s = \frac{1}{2}$$

بنابراین بیشینه مشتق سیگموید زمانی رخ می دهد که:

$$\sigma(z) = \frac{1}{2}$$

از آنجا که:

$$\sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2}$$

پس نقطه متناظر آن:



$$z = 0$$

است.

حال مقدار بیشینه مشتق:

$$\sigma'(z)_{\max} = \frac{1}{2} \left(1 - \frac{1}{2}\right)$$

$$= \frac{1}{2} \times \frac{1}{2}$$

$$= \frac{1}{4}$$

$$= 0.25$$

نتیجه بخش الف

$$\max_z \sigma'(z) = 0.25$$

این مقدار در نقطه زیر رخ می دهد:

$$z = 0$$

زیرا در این نقطه داریم:

$$\sigma(0) = 0.5$$

بنابراین مشتق تابع سیگموید هیچ گاه از 0.25 بیشتر نمی شود:

$$0 < \sigma'(z) \leq 0.25$$

۳.۲ بخش ب: بررسی سیگنال خطا و اثبات محو شدن گرادیان

در شبکه های عصبی، برای به روزرسانی وزن های لایه های ابتدایی باید گرادیان تابع هزینه نسبت به پارامترهای همان لایه ها محاسبه شود. این کار با استفاده از الگوریتم پس انتشار خطا یا Backpropagation انجام می شود.



۱.۳.۲ رابطه بازگشتی سیگنال خطا

سیگنال خطا در لایه k -ام برابر است با:

$$\delta^{(k)} = \frac{\partial L}{\partial z^{(k)}}$$

طبق قاعده زنجیره‌ای، رابطه بین خطای لایه k -ام و خطای لایه بعدی یعنی $k+1$ -ام به صورت زیر است:

$$\delta^{(k)} = \left(W^{(k+1)}\right)^T \delta^{(k+1)} \odot \sigma'(z^{(k)})$$

که در آن:

• $\left(W^{(k+1)}\right)^T$ ترانپوز ماتریس وزن لایه بعدی است.

• $\delta^{(k+1)}$ سیگنال خطای لایه بعدی است.

• $\sigma'(z^{(k)})$ مشتق تابع فعال‌ساز در لایه k -ام است.

• $\odot$ ضرب عضو به عضو است.

رابطه اصلی پس انتشار خطا

$$\delta^{(k)} = \left(W^{(k+1)}\right)^T \delta^{(k+1)} \odot \sigma'(z^{(k)})$$

این رابطه نشان می‌دهد که خطای هر لایه از حاصل ضرب خطای لایه بعدی، وزن‌ها و مشتق تابع فعال‌ساز به دست می‌آید.

۲.۳.۲ وابستگی خطا به حاصل ضرب مشتقات

اگر رابطه بالا را برای چند لایه پشت سر هم بنویسیم، خواهیم داشت:

$$\delta^{(L-1)} = \left(W^{(L)}\right)^T \delta^{(L)} \odot \sigma'(z^{(L-1)})$$

برای لایه $L-2$:

$$\delta^{(L-2)} = \left(W^{(L-1)}\right)^T \delta^{(L-1)} \odot \sigma'(z^{(L-2)})$$

اگر $\delta^{(L-1)}$ را جایگذاری کنیم، مشاهده می‌شود که $\delta^{(L-2)}$ شامل ضرب مشتقات دو لایه می‌شود.

به همین ترتیب، برای لایه اول، خطا به صورت کلی وابسته به ضرب زنجیره‌ای زیر است:

$$\delta^{(1)} \propto \left[\prod_{k=1}^{L-1} \sigma'(z^{(k)}) \right] \delta^{(L)}$$

البته در حالت دقیق‌تر، وزن‌ها نیز در این حاصل ضرب حضور دارند:

$$\delta^{(1)} = \left[\prod_{k=1}^{L-1} (W^{(k+1)})^T D^{(k)} \right] \delta^{(L)}$$

که در آن:

$$D^{(k)} = \text{diag}(\sigma'(z^{(k)}))$$

نکته بسیار مهم

در پس‌انتشار، خطا برای رسیدن به لایه‌های ابتدایی باید از مسیر چندین مشتق عبور کند. اگر مشتق‌ها کوچک باشند، حاصل ضرب آن‌ها به سرعت کوچک می‌شود.

۳.۳.۲ فرض نرمال بودن وزن‌ها

طبق صورت سوال، فرض می‌کنیم اندازه وزن‌ها حدوداً ۱ باشد. یعنی اثر وزن‌ها در بزرگ یا کوچک کردن شدید گرادیان را نادیده می‌گیریم و تقریباً داریم:

$$\|W^{(k)}\| \approx 1$$

در نتیجه عامل اصلی کوچک شدن گرادیان، مشتق تابع سیگموید است. از بخش قبل می‌دانیم:

$$\sigma'(z) \leq 0.25$$

بنابراین برای هر لایه داریم:

$$\|\sigma'(z^{(k)})\| \leq 0.25$$

حال با استفاده از رابطه بازگشتی خطا:

$$\delta^{(k)} = (W^{(k+1)})^T \delta^{(k+1)} \odot \sigma'(z^{(k)})$$



از دو طرف نرم می‌گیریم:

$$\|\delta^{(k)}\| = \left\| \left(W^{(k+1)} \right)^T \delta^{(k+1)} \odot \sigma'(z^{(k)}) \right\|$$

با استفاده از نامساوی نرم‌ها:

$$\|\delta^{(k)}\| \leq \left\| \left(W^{(k+1)} \right)^T \right\| \|\delta^{(k+1)}\| \|\sigma'(z^{(k)})\|$$

با فرض:

$$\left\| \left(W^{(k+1)} \right)^T \right\| \approx 1$$

و چون:

$$\|\sigma'(z^{(k)})\| \leq 0.25$$

پس:

$$\|\delta^{(k)}\| \leq 0.25 \|\delta^{(k+1)}\|$$

یک گام پس‌انتشار

$$\boxed{\|\delta^{(k)}\| \leq 0.25 \|\delta^{(k+1)}\|}$$

یعنی وقتی خطای لایه به عقب برمی‌گردد، اندازه آن حداکثر در 0.25 ضرب می‌شود.

۴.۳.۲ اثبات نامساوی برای لایه اول

اکنون رابطه بالا را چند بار پشت سر هم اعمال می‌کنیم.

برای لایه $L - 1$:

$$\|\delta^{(L-1)}\| \leq 0.25 \|\delta^{(L)}\|$$

برای لایه $L - 2$:

$$\|\delta^{(L-2)}\| \leq 0.25 \|\delta^{(L-1)}\|$$

با جایگذاری رابطه قبل:

$$\|\delta^{(L-2)}\| \leq 0.25 \times 0.25 \|\delta^{(L)}\|$$

$$\|\delta^{(L-2)}\| \leq (0.25)^2 \|\delta^{(L)}\|$$

به همین ترتیب، برای لایه اول باید $L - 1$ بار این ضرب انجام شود. بنابراین:

$$\|\delta^{(1)}\| \leq (0.25)^{L-1} \|\delta^{(L)}\|$$

نتیجه اصلی بخش ب

$$\|\delta^{(1)}\| \leq (0.25)^{L-1} \|\delta^{(L)}\|$$

این رابطه نشان می‌دهد که اگر تعداد لایه‌ها زیاد شود، خطای لایه‌های ابتدایی به صورت نمایی کوچک می‌شود.

۴.۲ بخش ج: تحلیل اثر افزایش تعداد لایه‌ها بر سرعت یادگیری

نتیجه بخش قبل نشان داد:

$$\|\delta^{(1)}\| \leq (0.25)^{L-1} \|\delta^{(L)}\|$$

از آنجا که:

$$0.25 < 1$$

وقتی L افزایش پیدا کند، مقدار زیر با سرعت زیادی به صفر نزدیک می‌شود:

$$(0.25)^{L-1}$$

برای مثال:

$$L = 2 \quad \Rightarrow \quad (0.25)^1 = 0.25$$



$$L = 5 \quad \Rightarrow \quad (0.25)^4 = 0.00390625$$

$$L = 10 \quad \Rightarrow \quad (0.25)^9 \approx 3.8 \times 10^{-6}$$

یعنی با افزایش تعداد لایه‌ها، گرادیانی که به لایه‌های ابتدایی می‌رسد، بسیار کوچک می‌شود.

مفهوم محو شدن گرادیان

وقتی گرادیان در لایه‌های ابتدایی بسیار کوچک شود، وزن‌های آن لایه‌ها تقریباً به‌روزرسانی نمی‌شوند. در نتیجه، لایه‌های ابتدایی شبکه بسیار کند یاد می‌گیرند یا عملاً یادگیری آن‌ها متوقف می‌شود. به این پدیده محو شدن گرادیان یا Vanishing Gradient گفته می‌شود.

۱.۴.۲ اثر روی به‌روزرسانی وزن‌ها

در الگوریتم گرادیان کاهشی، وزن‌ها طبق رابطه زیر به‌روزرسانی می‌شوند:

$$W_{\text{new}}^{(k)} = W_{\text{old}}^{(k)} - \eta \frac{\partial L}{\partial W^{(k)}}$$

که در آن η نرخ یادگیری است.

گرادیان وزن‌های لایه k -ام معمولاً به‌صورت زیر است:

$$\frac{\partial L}{\partial W^{(k)}} = \delta^{(k)} (a^{(k-1)})^T$$

پس اگر $\delta^{(k)}$ کوچک شود، گرادیان وزن‌ها نیز کوچک می‌شود:

$$\delta^{(k)} \approx 0 \quad \Rightarrow \quad \frac{\partial L}{\partial W^{(k)}} \approx 0$$

در نتیجه:

$$W_{\text{new}}^{(k)} \approx W_{\text{old}}^{(k)}$$

یعنی وزن‌های آن لایه تقریباً تغییری نمی‌کنند.



تحلیل نهایی اثر روی یادگیری

در لایه‌های انتهایی شبکه، گرادیان هنوز نسبتاً بزرگ است؛ اما هرچه به سمت لایه‌های ابتدایی‌تر می‌رویم، گرادیان در مشتقات سیگموید ضرب می‌شود و کوچک‌تر می‌گردد. بنابراین لایه‌های اولیه کندتر از لایه‌های پایانی یاد می‌گیرند.

۵.۲ جمع‌بندی نهایی سوال ۲

در این سوال نشان دادیم که تابع سیگموید اگرچه تابعی نرم و مشتق‌پذیر است، اما مشتق آن حداکثر برابر با ۰.۲۵ است:

$$\sigma'(z) \leq 0.25$$

از طرفی در شبکه‌های عمیق، گرادیان برای رسیدن به لایه‌های ابتدایی باید از چندین لایه عبور کند. طبق قاعدهٔ زنجیره‌ای، این عبور باعث ضرب شدن گرادیان در مشتق‌های متوالی تابع فعال‌ساز می‌شود. با فرض اینکه اندازهٔ وزن‌ها حدوداً ۱ باشد، داشتیم:

$$\|\delta^{(1)}\| \leq (0.25)^{L-1} \|\delta^{(L)}\|$$

پاسخ نهایی فشرده

$$\max_z \sigma'(z) = 0.25$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\delta^{(k)} = \left(W^{(k+1)}\right)^T \delta^{(k+1)} \odot \sigma'(z^{(k)})$$

$$\|\delta^{(1)}\| \leq (0.25)^{L-1} \|\delta^{(L)}\|$$

بنابراین با افزایش تعداد لایه‌ها، گرادیان لایه‌های ابتدایی به‌صورت نمایی کوچک می‌شود و سرعت یادگیری آن‌ها کاهش می‌یابد. این همان پدیدهٔ محو شدن گرادیان است.



سوال ۳

۱.۳ صورت مسئله و مفهوم قضیه تقریب عمومی

قضیه تقریب عمومی یا Universal Approximation Theorem بیان می‌کند که یک شبکه پرسپترون چندلایه با تنها یک لایه پنهان، اگر تعداد نوروهای کافی داشته باشد، می‌تواند هر تابع پیوسته روی یک مجموعه فشرده را با دقت دلخواه تقریب بزند. به صورت شهودی، اگر داشته باشیم:

$$f: \mathcal{K} \subset \mathbb{R}^n \rightarrow \mathbb{R}$$

که در آن $\mathcal{K}$ یک مجموعه فشرده باشد، آنگاه یک شبکه عصبی با یک لایه پنهان می‌تواند تابع f را به صورت زیر تقریب بزند:

$$\hat{f}(x) = \sum_{i=1}^m \alpha_i \sigma(w_i^T x + b_i)$$

به طوری که برای هر $\varepsilon > 0$ ، تعداد نوروهای m به اندازه کافی بزرگ انتخاب شود تا داشته باشیم:

$$|f(x) - \hat{f}(x)| < \varepsilon$$

برای همه x های موجود در دامنه مورد نظر.

نکته کلیدی درباره قضیه تقریب عمومی

قضیه تقریب عمومی فقط وجود یک شبکه برای تقریب تابع را تضمین می‌کند؛ اما درباره موارد زیر چیزی نمی‌گوید:

- تعداد نوروهای لازم چقدر است؟
- آموزش شبکه چقدر سخت است؟
- آیا با داده محدود می‌توان به آن شبکه رسید؟
- آیا شبکه از نظر محاسباتی بهینه است؟
- آیا یادگیری آن در عمل پایدار و قابل انجام است؟

بنابراین اگرچه از نظر تئوری یک شبکه کم عمق و بسیار عریض می‌تواند توابع زیادی را تقریب بزند، اما در عمل معمولاً شبکه‌های عمیق‌تر بسیار کارآمدتر و قابل آموزش‌تر هستند.



۲.۳ بخش الف: چرا به جای شبکه‌های بسیار عریض، از شبکه‌های عمیق استفاده می‌کنیم؟
بر اساس قضیه تقریب عمومی، ممکن است این سؤال ایجاد شود که اگر یک شبکه با یک لایه پنهان می‌تواند هر تابع پیوسته‌ای را تقریب بزند، پس چرا از شبکه‌های عمیق استفاده می‌کنیم؟
پاسخ اصلی این است که:

کارایی، یادگیری پذیری و بیان فشرده $\neq$ تقریب پذیری
یعنی اینکه یک شبکه بتواند از نظر تئوری تابعی را تقریب بزند، به این معنا نیست که این کار را با تعداد پارامتر کم، آموزش آسان و تعمیم مناسب انجام می‌دهد.

ایده اصلی بخش الف

قضیه تقریب عمومی می‌گوید شبکه کم عمق می‌تواند تقریب بزند، اما تضمین نمی‌کند که این تقریب با تعداد نوروں کم، آموزش راحت یا عملکرد خوب روی داده‌های جدید انجام شود.

۱.۲.۳. ۱. تفاوت بین توانایی وجودی و کارایی عملی

قضیه تقریب عمومی یک نتیجه وجودی است. یعنی فقط می‌گوید:

شبکه‌ای با یک لایه پنهان که تابع را تقریب بزند $\exists$

اما نمی‌گوید آن شبکه از نظر عملی مناسب است.

ممکن است برای تقریب یک تابع پیچیده، شبکه تک لایه پنهان به تعداد بسیار زیادی نوروں نیاز داشته باشد:

$$m \gg 1$$

در این حالت تعداد پارامترهای شبکه بسیار زیاد می‌شود و آموزش آن سخت خواهد شد.

برای مثال اگر ورودی $x \in \mathbb{R}^n$ باشد و یک لایه پنهان با m نوروں داشته باشیم، تعداد پارامترهای لایه پنهان تقریباً برابر است با:

$$m(n+1)$$

و اگر m بسیار بزرگ باشد، تعداد پارامترها نیز بسیار زیاد می‌شود.



مشکل شبکه بسیار عریض

شبکه بسیار عریض ممکن است از نظر تئوری قادر به تقریب تابع باشد، اما به دلیل تعداد زیاد پارامترها:

- حافظه زیادی مصرف می کند.
- آموزش آن کندتر می شود.
- احتمال بیش برآزش یا Overfitting افزایش می یابد.
- نیاز به داده آموزشی بیشتری پیدا می کند.

۲.۲.۳. بیان پذیری بهتر شبکه های عمیق

شبکه های عمیق به دلیل داشتن چندین لایه، می توانند توابع پیچیده را به صورت ترکیبی یا سلسله مراتبی نمایش دهند.

در بسیاری از مسائل واقعی، تابع هدف یک ساختار ترکیبی دارد. یعنی خروجی نهایی از ترکیب چندین ویژگی ساده تر ساخته می شود.
برای مثال در بینایی ماشین:

تشخیص شیء → اجزای شیء → شکل های ساده → لبه ها → پیکسل ها

در اینجا هر لایه یک سطح از ویژگی ها را یاد می گیرد:

$a^{(1)}$ = ویژگی های ساده

$a^{(2)}$ = ترکیب ویژگی های ساده

$a^{(3)}$ = ویژگی های پیچیده تر

$a^{(L)}$ = نمایش نهایی

بنابراین شبکه عمیق می تواند تابع پیچیده را به شکل ترکیب چند تابع ساده تر بنویسد:

$$f(x) = f_L(f_{L-1}(\dots f_2(f_1(x))))$$



مزیت بیان‌پذیری عمق

عمق باعث می‌شود شبکه بتواند نمایش‌های سلسله‌مراتبی بسازد. در بسیاری از مسائل، این نمایش سلسله‌مراتبی باعث می‌شود همان تابع با تعداد پارامتر بسیار کمتری نسبت به یک شبکه بسیار عریض تقریب زده شود.

۳.۲.۳. کاهش تعداد پارامترها

یکی از دلایل اصلی استفاده از شبکه‌های عمیق، کاهش تعداد پارامترهای موردنیاز برای نمایش یک تابع پیچیده است.

فرض کنید تابع هدف از ترکیب چند زیرتابع ساده ساخته شده باشد:

$$f(x) = g_3(g_2(g_1(x)))$$

یک شبکه عمیق می‌تواند هر کدام از این بخش‌ها را در یک لایه یا چند لایه مدل کند:

$$1 \text{ Layer} \approx g_1$$

$$2 \text{ Layer} \approx g_2$$

$$3 \text{ Layer} \approx g_3$$

اما یک شبکه کم‌عمق باید کل ترکیب را یک‌جا یاد بگیرد. این کار ممکن است به تعداد بسیار زیادی نورون نیاز داشته باشد.

شبکه کم‌عمق: تعداد نورون زیاد

شبکه عمیق: تعداد لایه بیشتر ولی نورون کمتر در هر لایه

مقایسه تعداد پارامترها

در بسیاری از توابع ساختاریافته، شبکه عمیق می‌تواند با تعداد پارامتر چندجمله‌ای، تابع را تقریب بزند؛ درحالی‌که شبکه کم‌عمق ممکن است برای همان دقت به تعداد پارامتری نیاز داشته باشد.

۴.۲.۳. یادگیری ویژگی‌ها به صورت سلسله‌مراتبی

یکی از مهم‌ترین مزیت‌های شبکه‌های عمیق این است که ویژگی‌ها را به صورت سلسله‌مراتبی یاد می‌گیرند. در یک شبکه عمیق:

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \dots \rightarrow h^{(L)}$$

لایه‌های ابتدایی معمولاً ویژگی‌های ساده‌تر را استخراج می‌کنند، و لایه‌های بعدی این ویژگی‌ها را ترکیب کرده و ویژگی‌های پیچیده‌تر می‌سازند. برای مثال در پردازش تصویر:

لایه‌های ابتدایی: لبه‌ها و گوشه‌ها

لایه‌های میانی: الگوها و بافت‌ها

لایه‌های عمیق‌تر: اجزای شیء و مفهوم کلی

این ساختار باعث می‌شود یادگیری تابع هدف طبیعی‌تر و کارآمدتر انجام شود.

نکته مهم

شبکه عمیق صرفاً تعداد لایه‌ها را زیاد نمی‌کند؛ بلکه امکان می‌دهد مدل، مسئله را مرحله‌به‌مرحله و در سطوح مختلف انتزاع یاد بگیرد.

۵.۲.۳. سهولت یادگیری و تعمیم بهتر

شبکه‌های بسیار عریض ممکن است به دلیل تعداد زیاد پارامترها، فضای جست‌وجوی بزرگی داشته باشند. در نتیجه، الگوریتم آموزش باید بین تعداد زیادی پارامتر به دنبال جواب مناسب بگردد. اما شبکه‌های عمیق با ساختار مناسب می‌توانند مسئله را به چند مرحله ساده‌تر تقسیم کنند. این موضوع باعث می‌شود آموزش در عمل بهتر انجام شود. همچنین اگر شبکه عمیق ساختار تابع واقعی را بهتر منعکس کند، معمولاً تعمیم بهتری روی داده‌های دیده‌نشده خواهد داشت.

تعمیم بهتر $\Rightarrow$ مدل مناسب‌تر

از دید یادگیری ماشین، هدف فقط کم کردن خطای آموزش نیست، بلکه باید خطای داده‌های جدید نیز کم باشد:

$$\text{Error Generalization} = \text{Error Test}$$

اگر شبکه کم‌عمق بسیار عریض باشد، ممکن است داده‌های آموزشی را حفظ کند، اما روی داده‌های جدید عملکرد ضعیف‌تری داشته باشد.

هشدار درباره شبکه بیش از حد عریض

تعداد زیاد نوروها و پارامترها می‌تواند ظرفیت مدل را بیش از حد افزایش دهد. اگر داده کافی یا منظم‌سازی مناسب وجود نداشته باشد، خطر بیش‌برازش افزایش می‌یابد.

۶.۲.۳ جمع‌بندی بخش الف

در نتیجه، با وجود قضیه تقریب عمومی، استفاده از شبکه‌های عمیق به دلایل زیر ترجیح داده می‌شود:

- شبکه عمیق می‌تواند توابع پیچیده را با تعداد پارامتر کمتر نمایش دهد.
- شبکه عمیق نمایش سلسله‌مراتبی از ویژگی‌ها می‌سازد.
- شبکه عمیق برای مسائل واقعی مثل تصویر، صوت و زبان طبیعی مناسب‌تر است.
- شبکه کم‌عمق ممکن است برای رسیدن به همان دقت به تعداد نمایی نورو نیاز داشته باشد.
- آموزش و تعمیم شبکه عمیق در بسیاری از کاربردها بهتر از شبکه بسیار عریض است.

پاسخ نهایی بخش الف

قضیه تقریب عمومی فقط تضمین می‌کند که یک شبکه تک‌لایه پنهان با نورو کافی می‌تواند تابع را تقریب بزند؛ اما درباره تعداد نوروها، تعداد پارامترها، سختی آموزش و توانایی تعمیم چیزی نمی‌گوید. شبکه‌های عمیق به دلیل ساختار سلسله‌مراتبی، بیان‌پذیری بهتر، تعداد پارامتر کمتر و یادگیری کارآمدتر، در عمل نسبت به شبکه‌های بسیار عریض ترجیح داده می‌شوند.

۳.۳ بخش ب: مثالی از تابعی که شبکه تک لایه برای نمایش آن به تعداد نمایی نوروں نیاز دارد

برای نشان دادن اهمیت عمق، می توان توابعی را در نظر گرفت که ساختار ترکیبی و سلسله مراتبی دارند. چنین توابعی را شبکه های عمیق به خوبی و با تعداد پارامتر کم نمایش می دهند؛ اما یک شبکه تک لایه پنهان برای نمایش آن ها ممکن است به تعداد بسیار زیادی نوروں نیاز داشته باشد. یک مثال مهم، توابعی هستند که از ترکیب تکراری عملیات ساده ساخته می شوند.

۱.۳.۳ مثال اول: تابع ضرب سلسله مراتبی

فرض کنید ورودی ها به صورت زیر باشند:

$$x = (x_1, x_2, \dots, x_{2d})$$

و تابع هدف برابر باشد با حاصل ضرب همه ورودی ها:

$$f(x) = x_1 x_2 x_3 \cdots x_{2d}$$

این تابع را می توان به صورت سلسله مراتبی محاسبه کرد. یعنی ابتدا ورودی ها را دوتادوتا ضرب می کنیم:

$$u_1 = x_1 x_2$$

$$u_2 = x_3 x_4$$

$$u_3 = x_5 x_6$$

و به همین ترتیب.

سپس خروجی های مرحله قبل دوباره دوتادوتا ضرب می شوند:

$$v_1 = u_1 u_2$$

$$v_2 = u_3 u_4$$

و این روند ادامه می یابد تا در نهایت کل حاصل ضرب به دست آید.



این ساختار را می‌توان به شکل یک درخت دودویی نمایش داد:

$$f(x) = \left(\left((x_1 x_2) (x_3 x_4) \right) \left((x_5 x_6) (x_7 x_8) \right) \dots \right)$$

ساختار سلسله‌مراتبی تابع ضرب

تابع ضرب تعداد زیادی ورودی را می‌توان با یک معماری عمیق به صورت مرحله‌ای محاسبه کرد. هر لایه فقط عملیات ساده ضرب روی خروجی‌های لایه قبل انجام می‌دهد.

۲.۳.۳ نمایش تابع ضرب با شبکه عمیق

در یک شبکه عمیق، هر بخش کوچک می‌تواند عمل ضرب دو مقدار را تقریب بزند:

$$g(a, b) \approx ab$$

سپس کل تابع به صورت ترکیب چندین تابع ساده نوشته می‌شود:

$$f(x) = g\left(g(x_1, x_2), g(x_3, x_4), \dots\right)$$

در این حالت عمق شبکه تقریباً برابر با تعداد مراحل ترکیب است:

$$d = \log_2 n$$

که در آن:

$$n = 2^d$$

است.

تعداد واحدهای لازم در هر مرحله متناسب با تعداد جفت‌ها است، و کل تعداد واحدها تقریباً از مرتبه چندجمله‌ای نسبت به n خواهد بود:

$$\text{poly}(n) \quad \text{یا در حالت کلی} \quad \mathcal{O}(n) = \text{تعداد نوروها در شبکه عمیق}$$

اما اگر بخواهیم همین تابع را با یک شبکه تک‌لایه پنهان نمایش دهیم، شبکه باید همه تعاملات مرتبه‌بالای بین متغیرها را یک‌جا مدل کند. این کار معمولاً به تعداد بسیار زیادی نرون نیاز دارد.



$\approx \mathcal{O}(2^n)$ تعداد نورون‌ها در شبکه کم عمق

مزیت عمق در تابع ضرب

شبکه عمیق می‌تواند تابع ضرب را با استفاده از ترکیب مرحله‌ای عملیات ساده نمایش دهد؛ اما شبکه تک‌لایه باید رابطه همه متغیرها را به صورت مستقیم و بدون ساختار میانی یاد بگیرد، که بسیار ناکارآمد است.

۳.۳.۳ مثال دوم: تابع ترکیبی سلسله‌مراتبی

یک مثال عمومی‌تر، تابعی است که به صورت ترکیب توابع ساده تعریف شود:

$$f(x_1, x_2, \dots, x_8) = g_3(g_2(g_1(x_1, x_2), g_1(x_3, x_4)), g_2(g_1(x_5, x_6), g_1(x_7, x_8)))$$

این تابع دارای ساختار سلسله‌مراتبی است.

در مرحله اول:

$$g_1(x_1, x_2), \quad g_1(x_3, x_4), \quad g_1(x_5, x_6), \quad g_1(x_7, x_8)$$

محاسبه می‌شود.

در مرحله دوم:

$$g_2(\cdot, \cdot)$$

روی خروجی‌های مرحله قبل اعمال می‌شود.

در مرحله سوم:

$$g_3(\cdot, \cdot)$$

خروجی نهایی را تولید می‌کند.

این دقیقاً همان چیزی است که شبکه‌های عمیق به خوبی انجام می‌دهند:

لایه اول: یادگیری روابط محلی



لایه دوم: ترکیب روابط محلی

لایه‌های بعدی: ساختن روابط پیچیده‌تر

مثال ساده از مزیت عمق

اگر تابع هدف به صورت ترکیب چند تابع ساده ساخته شده باشد، شبکه عمیق می‌تواند هر بخش را در یک لایه یاد بگیرد. اما شبکه کم عمق باید کل این ترکیب را به صورت یک جا نمایش دهد که معمولاً به تعداد بسیار بیشتری نرون نیاز دارد.

۴.۳.۳ تفاوت رشد چند جمله‌ای و نمایی

فرض کنید پیچیدگی مسئله با تعداد ورودی‌ها n افزایش یابد. در شبکه عمیق، اگر تابع ساختار سلسله مراتبی داشته باشد، تعداد نرون‌های لازم ممکن است به صورت چند جمله‌ای رشد کند:

$$\text{poly}(n)$$

برای مثال:

$$n^2, \quad n^3, \quad n^4$$

اما در شبکه تک لایه پنهان، تعداد نرون‌های مورد نیاز ممکن است به صورت نمایی رشد کند:

$$2^n$$

رشد نمایی بسیار سریع‌تر از رشد چند جمله‌ای است. برای مقایسه:

$$n = 10 \quad \Rightarrow \quad n^2 = 100$$

اما:

$$2^{10} = 1024$$

و برای:



$$n = 30$$

داریم:

$$n^2 = 900$$

ولی:

$$2^{30} = 1073741824$$

اهمیت تفاوت رشد‌ها

اگر تعداد نوروں‌های لازم به صورت نمایی رشد کند، مدل از نظر محاسباتی و حافظه‌ای غیرعملی می‌شود. عمق شبکه می‌تواند این رشد نمایی را در بسیاری از توابع ساختاریافته به رشد چندجمله‌ای کاهش دهد.

۴.۳ تحلیل ارتباط با قضیه تقریب عمومی

ممکن است به نظر برسد که مثال‌های بالا با قضیه تقریب عمومی تناقض دارند، اما چنین نیست. قضیه تقریب عمومی می‌گوید:

شبکه تک‌لایه پنهان می‌تواند تقریب بزند

اما نمی‌گوید:

با تعداد نوروں کم تقریب می‌زند

بنابراین ممکن است یک شبکه کم‌عمق بتواند تابعی را تقریب بزند، اما برای رسیدن به دقت مناسب به تعداد نمایی نوروں نیاز داشته باشد.

از طرف دیگر، شبکه عمیق با استفاده از ساختار ترکیبی تابع، همان تابع را با تعداد بسیار کمتری نوروں تقریب می‌زند.

شبکه کم‌عمق: تقریب ممکن، اما ناکارآمد

شبکه عمیق: تقریب کارآمد و ساختاریافته

رفع ابهام مهم

قضیه تقریب عمومی با مزیت شبکه‌های عمیق تناقض ندارد. این قضیه درباره امکان تقریب صحبت می‌کند، اما مزیت عمق مربوط به کارایی تقریب، تعداد پارامترها، ساختار تابع و سهولت یادگیری است.

۵.۳ جمع‌بندی نهایی سوال ۳

در این سوال بررسی کردیم که اگرچه طبق قضیه تقریب عمومی، یک شبکه عصبی با یک لایه پنهان و تعداد کافی نرون می‌تواند هر تابع پیوسته روی یک بازه فشرده را تقریب بزند، اما این موضوع به تنهایی دلیل کافی برای استفاده از شبکه‌های بسیار عریض نیست. شبکه‌های عمیق در عمل مزایای مهم‌تری دارند:

- نمایش توابع پیچیده با تعداد پارامتر کمتر
- توانایی یادگیری ویژگی‌های سلسله‌مراتبی
- مناسب بودن برای داده‌های ساختاریافته مانند تصویر، صوت و متن
- کاهش نیاز به تعداد نمایی نرون‌ها
- امکان یادگیری کارآمدتر و تعمیم بهتر
- همچنین دیدیم که توابع ترکیبی مانند:

$$f(x) = x_1 x_2 \cdots x_n$$

یا توابعی از فرم:

$$f(x) = f_L(f_{L-1}(\cdots f_1(x)))$$

می‌توانند توسط شبکه‌های عمیق با تعداد پارامتر چندجمله‌ای تقریب زده شوند، درحالی‌که شبکه‌های تک‌لایه پنهان ممکن است برای تقریب همان توابع به تعداد نمایی نرون نیاز داشته باشند.

پاسخ نهایی فشرده

قضیه تقریب عمومی فقط بیان می‌کند که شبکه تک‌لایه پنهان از نظر تئوری توانایی تقریب توابع پیوسته را دارد، اما درباره تعداد نرون‌های لازم، کارایی محاسباتی و سهولت آموزش چیزی نمی‌گوید. شبکه‌های عمیق به دلیل ساختار سلسله‌مراتبی و ترکیبی، می‌توانند بسیاری از توابع پیچیده را با تعداد پارامتر بسیار کمتر و یادگیری کارآمدتر نمایش دهند. برای مثال، توابعی مانند حاصل ضرب تعداد زیادی متغیر یا توابع ترکیبی سلسله‌مراتبی، با شبکه عمیق به صورت چندجمله‌ای قابل نمایش هستند، اما شبکه کم‌عمق ممکن است برای همان کار به تعداد نمایی نرون نیاز داشته باشد.

۴ سوال ۴

در این سوال، هدف بررسی روش‌های بهینه‌سازی پیشرفته در آموزش شبکه‌های عصبی است. در روش گرادین کاهشی معمولی، پارامترها طبق رابطه زیر به‌روزرسانی می‌شوند:

$$\theta_{t+1} = \theta_t - \alpha g_t$$

که در آن g_t گرادین تابع هزینه نسبت به پارامترها در گام t ، و α نرخ یادگیری است. مشکل اصلی این روش آن است که نرخ یادگیری برای همه جهت‌ها یکسان در نظر گرفته می‌شود و از اطلاعات گام‌های قبلی استفاده مؤثری نمی‌کند. الگوریتم‌های Momentum، RMSProp و Adam برای رفع این مشکلات معرفی شده‌اند.

۱.۴ بخش الف: الگوریتم Momentum

در روش Momentum، به جای استفاده مستقیم از گرادین فعلی، یک بردار سرعت یا مومنتوم تعریف می‌شود که میانگینی از گرادین‌های قبلی و گرادین فعلی است. روابط به‌روزرسانی به صورت زیر هستند:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

که در آن:

$$0 \leq \beta_1 < 1$$

پارامتر β_1 میزان حفظ اطلاعات گرادیان‌های گذشته را کنترل می‌کند. اگر β_1 بزرگ باشد، اثر گرادیان‌های قبلی بیشتر حفظ می‌شود و اگر کوچک باشد، الگوریتم بیشتر به گرادیان فعلی وابسته خواهد بود.

۱.۱.۴ تفسیر بردار سرعت

بردار v_t را می‌توان به عنوان یک میانگین متحرک نمایی از گرادیان‌ها در نظر گرفت. یعنی:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

اگر این رابطه را باز کنیم، خواهیم داشت:

$$v_t = (1 - \beta_1) g_t + \beta_1 (1 - \beta_1) g_{t-1} + \beta_1^2 (1 - \beta_1) g_{t-2} + \dots$$

بنابراین v_t تنها به گرادیان فعلی وابسته نیست، بلکه تاریخچه‌ای از گرادیان‌های قبلی را نیز در خود نگه می‌دارد. هرچه گرادیان قدیمی‌تر باشد، ضریب آن کوچک‌تر می‌شود.

$$\text{اثر گرادیان‌های قدیمی‌تر} \propto \beta_1^k$$

پس Momentum باعث می‌شود جهت حرکت شبکه فقط با یک گرادیان لحظه‌ای تعیین نشود، بلکه روند کلی حرکت نیز در نظر گرفته شود.

۲.۱.۴ اثر Momentum در راستای گرادیان‌های هم‌جهت

اگر گرادیان‌ها در چند گام متوالی تقریباً هم‌جهت باشند، یعنی:

$$g_t \approx g_{t-1} \approx g_{t-2}$$

آنگاه مقدار v_t در همان جهت تقویت می‌شود. در این حالت، الگوریتم در مسیر اصلی حرکت سرعت بیشتری پیدا می‌کند.

به بیان ساده، اگر شبکه در چند گام متوالی تشخیص دهد که باید در یک جهت مشخص حرکت کند، Momentum آن جهت را تقویت کرده و باعث شتاب گرفتن بهینه‌سازی می‌شود.

$$\text{افزایش سرعت در مسیر اصلی} \implies \text{گرادیان‌های هم‌جهت}$$

۳.۱.۴ اثر Momentum در راستای نوسان‌ها

در بسیاری از مسائل، سطح تابع هزینه در بعضی جهت‌ها دارای نوسان زیاد است. یعنی گرادیان‌ها در یک جهت مرتباً علامت عوض می‌کنند:

$$g_t \approx -g_{t-1}$$

در چنین حالتی، جمله‌های مربوط به گرادیان‌های قبلی و فعلی تا حدی یکدیگر را خنثی می‌کنند. بنابراین مقدار v_t در راستاهای نوسانی کوچک می‌شود. پس Momentum باعث می‌شود:

نوسان‌های شدید کاهش یابد

و حرکت بهینه‌سازی نرم‌تر و پایدارتر شود.

جمع‌بندی بخش Momentum

الگوریتم Momentum با استفاده از میانگین متحرک گرادیان‌ها، در جهت‌هایی که گرادیان‌ها هم‌راستا هستند سرعت حرکت را افزایش می‌دهد و در جهت‌هایی که گرادیان‌ها نوسانی هستند، نوسان‌ها را کاهش می‌دهد. در نتیجه، مسیر حرکت به سمت کمینه پایدارتر و سریع‌تر می‌شود.

۲.۴ بخش ب: الگوریتم RMSProp

در برخی مسائل، مقدار گرادیان در راستاهای مختلف بسیار متفاوت است. برای مثال ممکن است در یک جهت گرادیان بسیار بزرگ و در جهت دیگر بسیار کوچک باشد. اگر برای همه پارامترها از یک نرخ یادگیری یکسان استفاده شود، حرکت در جهت‌هایی با گرادیان بزرگ ناپایدار شده و در جهت‌هایی با گرادیان کوچک بسیار کند می‌شود.

الگوریتم RMSProp برای حل این مشکل، نرخ یادگیری را برای هر پارامتر به صورت جداگانه تنظیم می‌کند. روابط آن به صورت زیر هستند:

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}$$



که در آن:

$$0 \leq \beta_2 < 1$$

و ϵ عددی بسیار کوچک و مثبت است که برای جلوگیری از تقسیم بر صفر استفاده می‌شود.

۱.۲.۴ تفسیر جمله s_t

در رابطه بالا، g_t^2 به صورت مؤلفه به مؤلفه محاسبه می‌شود. یعنی اگر گرادیان به صورت برداری باشد:

$$g_t = \begin{bmatrix} g_{t,1} \\ g_{t,2} \\ \vdots \\ g_{t,n} \end{bmatrix}$$

آنگاه:

$$g_t^2 = \begin{bmatrix} g_{t,1}^2 \\ g_{t,2}^2 \\ \vdots \\ g_{t,n}^2 \end{bmatrix}$$

بنابراین s_t میانگین متحرک نمایی مربع گرادیان‌ها است. اگر در یک مؤلفه، گرادیان‌ها معمولاً بزرگ باشند، مقدار متناظر در s_t بزرگ می‌شود. اگر گرادیان‌ها کوچک باشند، مقدار متناظر در s_t کوچک می‌ماند.

$$s_t \approx \mathbb{E}[g_t^2]$$

۲.۲.۴ تنظیم تطبیقی نرخ یادگیری

در RMSProp، گرادیان بر $\sqrt{s_t} + \epsilon$ تقسیم می‌شود:

$$\frac{g_t}{\sqrt{s_t} + \epsilon}$$

این عبارت باعث می‌شود اندازه گام به صورت تطبیقی تنظیم شود. اگر در یک جهت گرادیان‌ها بزرگ باشند، آنگاه:

بزرگ می شود s_t

در نتیجه مخرج کسر بزرگ تر شده و اندازه گام کاهش می یابد:

$$\frac{g_t}{\sqrt{s_t} + \epsilon} \text{ کوچک تر می شود}$$

پس الگوریتم از حرکت های بسیار بزرگ و ناپایدار جلوگیری می کند. از طرف دیگر، اگر در یک جهت گرادیان ها کوچک باشند، مقدار s_t نیز کوچک تر است. بنابراین مخرج کسر کوچک تر شده و کاهش بیش از حد اندازه گام اتفاق نمی افتد. در نتیجه الگوریتم می تواند در جهت هایی که گرادیان کوچک است همچنان حرکت مؤثری داشته باشد.

گام کوچک تر $\Rightarrow$ گرادیان بزرگ

گام نسبتاً بزرگ تر $\Rightarrow$ گرادیان کوچک

۳.۲.۴ اثر RMSProp در دره های باریک تابع هزینه

در بسیاری از توابع هزینه، سطح خطا شبیه یک دره باریک است. در یک جهت، شیب بسیار زیاد است و در جهت دیگر، شیب کم است. روش گرادیان کاهشی معمولی ممکن است در جهت شیب زیاد دچار نوسان شود و در جهت اصلی حرکت به کندی پیش برود.

RMSProp در جهت هایی که گرادیان بزرگ است، نرخ یادگیری مؤثر را کاهش می دهد. بنابراین نوسان ها در راستاهای پرشیب کم می شوند. همچنین در جهت هایی که گرادیان کوچک است، اجازه می دهد حرکت همچنان ادامه پیدا کند.

$$\alpha_{\text{effective},i} = \frac{\alpha}{\sqrt{s_{t,i}} + \epsilon}$$

پس برای مؤلفه i -ام داریم:

$$\theta_{t+1,i} = \theta_{t,i} - \alpha_{\text{effective},i} g_{t,i}$$

در نتیجه RMSProp برای هر پارامتر یک نرخ یادگیری مؤثر جداگانه ایجاد می کند.



جمع‌بندی بخش RMSProp

الگوریتم RMSProp با نگهداری میانگین متحرک مربع گرادیان‌ها، اندازه گام را در هر راستا تنظیم می‌کند. در راستاهایی با گرادیان بزرگ، گام کوچک‌تر می‌شود و از نوسان و ناپایداری جلوگیری می‌کند. در راستاهایی با گرادیان کوچک، گام نسبی بزرگ‌تر باقی می‌ماند و سرعت همگرایی افزایش می‌یابد.

۳.۴ بخش ج: الگوریتم Adam

الگوریتم Adam یا Adaptive Moment Estimation ترکیبی از ایده‌های Momentum و RMSProp است. این الگوریتم هم میانگین متحرک گرادیان‌ها را نگه می‌دارد و هم میانگین متحرک مربع گرادیان‌ها را. در Adam، دو کمیت اصلی تعریف می‌شود:

$$m_t$$

و

$$v_t$$

که در آن m_t تخمین ممان اول، یعنی میانگین گرادیان‌ها، و v_t تخمین ممان دوم، یعنی میانگین مربع گرادیان‌ها است. روابط اصلی الگوریتم Adam به صورت زیر هستند:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

در اینجا m_t مشابه Momentum عمل می‌کند و جهت کلی حرکت را هموار و پایدار می‌سازد. همچنین v_t مشابه RMSProp است و اندازه گام را برای هر پارامتر تنظیم می‌کند.

۱.۳.۴ اصلاح بایاس در Adam

در ابتدای آموزش معمولاً داریم:

$$m_0 = 0$$

$$v_0 = 0$$

به همین دلیل در گام‌های ابتدایی، مقادیر m_t و v_t به سمت صفر بایاس دارند؛ یعنی مقدار آن‌ها کمتر از مقدار واقعی میانگین گرادیان و مربع گرادیان تخمین زده می‌شود. برای رفع این مشکل، Adam از اصلاح بایاس استفاده می‌کند:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

در نتیجه تخمین‌های اصلاح‌شده $\hat{m}_t$ و $\hat{v}_t$ در گام‌های ابتدایی دقیق‌تر هستند.

۲.۳.۴ رابطه نهایی به‌روزرسانی در Adam

پس از محاسبه تخمین‌های اصلاح‌شده، پارامترها به‌صورت زیر به‌روزرسانی می‌شوند:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

این رابطه نشان می‌دهد که Adam از دو ایده به‌صورت هم‌زمان استفاده می‌کند:

$\hat{m}_t \implies$ جهت حرکت هموارشده مانند Momentum

$\sqrt{\hat{v}_t} \implies$ تنظیم اندازه گام مانند RMSProp

بنابراین Adam هم جهت حرکت را پایدار می‌کند و هم نرخ یادگیری مؤثر را برای هر پارامتر به‌صورت تطبیقی تنظیم می‌کند.

۳.۳.۴ نقش ابرپارامترهای Adam

الگوریتم Adam دارای چند ابرپارامتر مهم است:

$$\alpha, \beta_1, \beta_2, \epsilon$$

نقش هرکدام به صورت زیر است:

- α : نرخ یادگیری اصلی است و اندازه کلی گام‌ها را تعیین می‌کند.
- β_1 : میزان حافظهٔ ممان اول یا میانگین گرادیان‌ها را کنترل می‌کند. مقدار بزرگ‌تر باعث هموارسازی بیشتر جهت حرکت می‌شود.
- β_2 : میزان حافظهٔ ممان دوم یا میانگین مربع گرادیان‌ها را کنترل می‌کند. مقدار بزرگ‌تر باعث می‌شود تخمین اندازه گرادیان‌ها پایدارتر و کندتر تغییر کند.
- ϵ : عددی کوچک برای جلوگیری از تقسیم بر صفر و افزایش پایداری عددی است.

مقادیر رایج این پارامترها معمولاً به صورت زیر انتخاب می‌شوند:

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

$$\epsilon = 10^{-8}$$

—

۴.۳.۴ تغییر رفتار یادگیری با تنظیم پارامترها

اگر α بزرگ انتخاب شود، سرعت یادگیری زیاد می‌شود؛ اما ممکن است الگوریتم ناپایدار شود و از کمینه عبور کند:

سرعت بیشتر، اما احتمال نوسان بیشتر $\alpha \uparrow \Rightarrow$

اگر α کوچک باشد، آموزش پایدارتر است ولی ممکن است بسیار کند شود:

پایداری بیشتر، اما سرعت کمتر $\alpha \downarrow \Rightarrow$



اگر β_1 بزرگ‌تر شود، اثر گرادیان‌های قبلی بیشتر حفظ می‌شود. این کار باعث نرم‌تر شدن مسیر حرکت می‌شود، اما ممکن است واکنش الگوریتم به تغییرات جدید گرادیان کندتر شود:

$$\beta_1 \uparrow \implies \text{هموارسازی بیشتر جهت حرکت}$$

اگر β_2 بزرگ‌تر شود، تخمین v_t پایدارتر می‌شود، اما تغییرات ناگهانی در بزرگی گرادیان‌ها دیرتر در آن اثر می‌گذارد:

$$\beta_2 \uparrow \implies \text{تنظیم آرام‌تر و پایدارتر نرخ یادگیری}$$

اگر ϵ خیلی بزرگ انتخاب شود، اثر نرمال‌سازی توسط $\sqrt{v_t}$ کاهش می‌یابد. اگر خیلی کوچک باشد، ممکن است در برخی شرایط ناپایداری عددی رخ دهد.

۵.۳.۴ مقایسه Adam با Momentum و RMSProp

الگوریتم Momentum فقط از میانگین گرادیان‌ها استفاده می‌کند:

$$m_t \approx \text{میانگین گرادیان‌ها}$$

الگوریتم RMSProp فقط از میانگین مربع گرادیان‌ها استفاده می‌کند:

$$v_t \approx \text{میانگین مربع گرادیان‌ها}$$

اما Adam هر دو را ترکیب می‌کند:

$$\text{Adam} = \text{Momentum} + \text{RMSProp}$$

به همین دلیل Adam معمولاً در عمل یکی از پرکاربردترین الگوریتم‌های بهینه‌سازی در شبکه‌های عصبی است.

جمع‌بندی بخش Adam

الگوریتم Adam با ترکیب Momentum و RMSProp، هم جهت حرکت را با استفاده از میانگین گرادیان‌ها پایدار می‌کند و هم اندازه گام را با استفاده از میانگین مربع گرادیان‌ها برای هر پارامتر تنظیم می‌کند. همچنین با اصلاح بایاس، تخمین‌ها در گام‌های ابتدایی دقیق‌تر می‌شوند.



۴.۴ جمع بندی نهایی سوال ۴

در این سوال سه الگوریتم مهم بهینه سازی بررسی شدند.
در روش Momentum، از بردار سرعت استفاده می شود:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

این روش باعث می شود حرکت در جهت های پایدار و هم راستا تقویت شود و نوسان ها در جهت های متغیر کاهش یابد.

در روش RMSProp، میانگین متحرک مربع گرادیان ها محاسبه می شود:

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}$$

این روش نرخ یادگیری مؤثر را برای هر پارامتر تنظیم می کند و باعث کاهش نوسان در جهت های دارای گرادیان بزرگ و بهبود حرکت در جهت های دارای گرادیان کوچک می شود.
در نهایت، الگوریتم Adam ترکیبی از این دو روش است:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

پاسخ نهایی فشرده سوال ۴

الگوریتم Momentum با استفاده از میانگین متحرک گرادیان‌ها، حرکت را در جهت‌های اصلی شتاب می‌دهد و نوسان‌ها را کاهش می‌دهد. الگوریتم RMSProp با تقسیم گرادیان بر ریشه میانگین مربع گرادیان‌ها، نرخ یادگیری مؤثر را برای هر پارامتر تطبیق می‌دهد و از گام‌های بسیار بزرگ یا بسیار کوچک جلوگیری می‌کند. الگوریتم Adam این دو ایده را ترکیب می‌کند؛ یعنی هم از ممان اول گرادیان برای تعیین جهت پایدار حرکت استفاده می‌کند و هم از ممان دوم برای تنظیم اندازه گام‌ها. بنابراین Adam معمولاً سریع، پایدار و مناسب برای آموزش شبکه‌های عصبی عمیق است.

۵ پیش‌بینی قیمت مسکن

Q۵ Notebook Colab Google

در این سوال قصد داریم با استفاده از مجموعه داده واقعی Ames Housing مربوط به شهر Ames در ایالت Iowa، یک مدل مبتنی بر شبکه عصبی چندلایه (MLP) برای پیش‌بینی قیمت فروش خانه‌ها طراحی و پیاده‌سازی کنیم. این مجموعه داده شامل اطلاعات مختلفی درباره ویژگی‌های فیزیکی و محیطی خانه‌ها است. هدف این مسئله پیش‌بینی مقدار SalePrice می‌باشد که یک مسئله از نوع Regression است.

۱.۵ (الف) بررسی اولیه داده

۱.۱.۵ بارگذاری داده و نمایش ۱۰ سطر اول

در اولین گام، فایل AmesHousing.csv با استفاده از کتابخانه pandas بارگذاری شده و ۱۰ سطر ابتدایی آن برای آشنایی با ساختار ویژگی‌ها نمایش داده می‌شود.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Load dataset
6 df = pd.read_csv("AmesHousing.csv")
7
8 # Display first 10 rows
9 print(df.head(10))
10
```

Listing 1: Loading dataset and displaying initial rows



Order	PID	MS	SubClass	MS	Zoning	Lot	Frontage	Lot	Area	Street	\
0	1	526301100		20	RL	141.0		31770		Pave	
1	2	526350040		20	RH	80.0		11622		Pave	
2	3	526351010		20	RL	81.0		14267		Pave	
3	4	526353030		20	RL	93.0		11160		Pave	
4	5	527105010		60	RL	74.0		13830		Pave	
5	6	527105030		60	RL	78.0		9978		Pave	
6	7	527127150		120	RL	41.0		4920		Pave	
7	8	527145080		120	RL	43.0		5005		Pave	
8	9	527146030		120	RL	39.0		5389		Pave	
9	10	527162130		60	RL	60.0		7500		Pave	

Alley	Lot	Shape	Land	Contour	...	Pool	Area	Pool	QC	Fence	Misc	Feature	\
0	NaN	IR1		Lvl	...	0	NaN	NaN		NaN		NaN	
1	NaN	Reg		Lvl	...	0	NaN	MnPrv		NaN		NaN	
2	NaN	IR1		Lvl	...	0	NaN	NaN		NaN		Gar2	
3	NaN	Reg		Lvl	...	0	NaN	NaN		NaN		NaN	
4	NaN	IR1		Lvl	...	0	NaN	MnPrv		NaN		NaN	
5	NaN	IR1		Lvl	...	0	NaN	NaN		NaN		NaN	
6	NaN	Reg		Lvl	...	0	NaN	NaN		NaN		NaN	
7	NaN	IR1		HLS	...	0	NaN	NaN		NaN		NaN	
8	NaN	IR1		Lvl	...	0	NaN	NaN		NaN		NaN	
9	NaN	Reg		Lvl	...	0	NaN	NaN		NaN		NaN	

Misc	Val	Mo	Sold	Yr	Sold	Sale	Type	Sale	Condition	SalePrice
0	0		5	2010		WD		Normal		215000
1	0		6	2010		WD		Normal		105000
2	12500		6	2010		WD		Normal		172000
3	0		4	2010		WD		Normal		244000
4	0		3	2010		WD		Normal		189900
5	0		6	2010		WD		Normal		195500
6	0		4	2010		WD		Normal		213500
7	0		1	2010		WD		Normal		191500
8	0		3	2010		WD		Normal		236500
9	0		6	2010		WD		Normal		189000

[10 rows x 82 columns]

شکل ۱: نمای کلی از ۱۰ سطر اول مجموعه داده

۲.۱.۵ گزارش تعداد نمونه‌ها و ویژگی‌ها

با بررسی ابعاد دیتاست، تعداد کل خانه‌های ثبت شده (نمونه‌ها) و تعداد متغیرهای اندازه‌گیری شده (ویژگی‌ها) استخراج می‌گردد.

ابعاد مجموعه داده

تعداد کل نمونه‌ها (Samples): ۲۹۳۰

تعداد کل ویژگی‌ها (Features): ۸۲

۳.۱.۵ بررسی ستون‌های دارای مقادیر گم‌شده (Missing Values)

در این بخش، تمام ستون‌هایی که حداقل یک مقدار NaN دارند شناسایی شده و تعداد داده‌های گم‌شده در هر کدام گزارش می‌شود تا در مراحل پیش‌پردازش مورد استفاده قرار گیرد.

```
# Check for missing values
```

```

2     missing_counts = df.isnull().sum()
3     missing_columns = missing_counts[missing_counts > 0].sort_values(
ascending=False)
4     print(missing_columns)
5

```

Listing 2: Identifying missing values

```

Pool QC      2917
Misc Feature 2824
Alley        2732
Fence        2358
Mas Vnr Type 1775
Fireplace Qu 1422
Lot Frontage 490
Garage Qual   159
Garage Cond   159
Garage Yr Blt 159
Garage Finish 159
Garage Type   157
Bsmt Exposure 83
BsmtFin Type 2 81
Bsmt Cond     80
Bsmt Qual     80
BsmtFin Type 1 80
Mas Vnr Area  23
Bsmt Full Bath 2
Bsmt Half Bath 2
BsmtFin SF 1   1
BsmtFin SF 2   1
Electrical     1
Total Bsmt SF  1
Bsmt Unf SF    1
Garage Area    1
Garage Cars    1
dtype: int64

```

شکل ۲: ستون‌هایی که دارای داده‌های تهی هستند

۴.۱.۵ رسم هیستوگرام متغیر هدف و گزارش میانگین

متغیر هدف ما قیمت فروش (SalePrice) است. بررسی توزیع این متغیر و محاسبه میانگین آن، دید اولیه‌ای از بازه قیمتی خانه‌ها به ما می‌دهد.

```

1     # Mean of SalePrice
2     mean_price = df['SalePrice'].mean()
3     print(f"Mean SalePrice: {mean_price:.2f}")
4
5     # Histogram of SalePrice
6     plt.figure(figsize=(10, 6))
7     sns.histplot(df['SalePrice'], kde=True, color='blue')
8     plt.title('Distribution of SalePrice')
9     plt.xlabel('Sale Price')
10    plt.ylabel('Frequency')

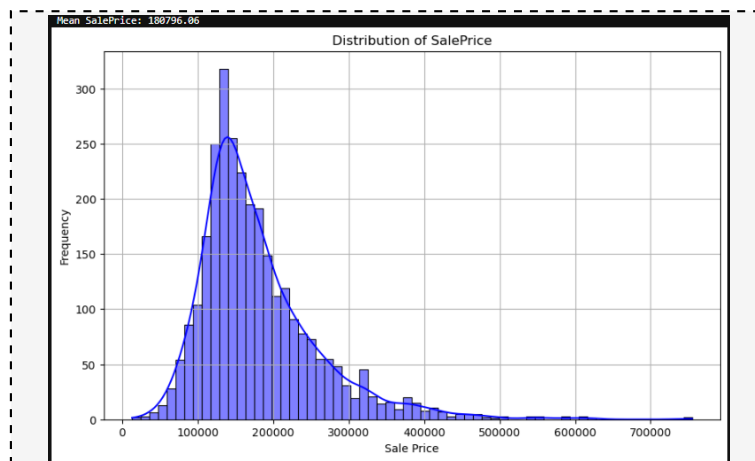
```

```
plt.grid(True)
plt.show()
```

Listing 3: Counting and reporting missing data

تحلیل آماری متغیر هدف

میانگین قیمت فروش خانه‌ها در این مجموعه داده برابر با 180796.06 دلار است. توزیع قیمت‌ها معمولاً دارای چولگی به راست است که نشان‌دهنده وجود خانه‌هایی با قیمت بسیار بالا نسبت به میانگین است.



شکل ۳: توزیع فراوانی قیمت فروش خانه‌ها

نتیجه‌گیری پارت الف

داده‌ها با موفقیت بارگذاری شدند. تعداد ویژگی‌های زیاد و وجود مقادیر گم‌شده در ستون‌هایی مانند Pool QC و Misc Feature نشان می‌دهد که بخش پیش‌پردازش داده‌ها (Preprocessing) در پارت‌های بعدی بسیار حیاتی خواهد بود.

۲.۵ (ب) پیش‌پردازش داده

۱.۲.۵ جایگزینی مقادیر گم‌شده

در این مرحله، داده‌های گم‌شده در ستون‌های عددی و کیفی به‌صورت مناسب جایگزین می‌شوند. برای ستون‌های عددی از میانگین ستون و برای ستون‌های کیفی از پرتکرارترین مقدار استفاده می‌شود.

```
import pandas as pd
```

```
2 import numpy as np
3
4 df = pd.read_csv("AmesHousing.csv")
5
6 # Separate numerical and categorical columns
7 num_cols = df.select_dtypes(include=[np.number]).columns
8 cat_cols = df.select_dtypes(exclude=[np.number]).columns
9
10 # Impute numerical columns with mean
11 for col in num_cols:
12     if df[col].isnull().sum() > 0:
13         df[col].fillna(df[col].mean(), inplace=True)
14
15 # Impute categorical columns with mode
16 for col in cat_cols:
17     if df[col].isnull().sum() > 0:
18         df[col].fillna(df[col].mode()[0], inplace=True)
19
```

Listing 4: Missing value imputation

نکته مهم

در این داده‌ست، ۳۸ ویژگی عددی و ۴۴ ویژگی کیفی وجود دارد. پس از انجام جایگزینی مقادیر گم‌شده، داده‌ها برای مرحله‌ی Encoding آماده می‌شوند.

۲.۲.۵ تبدیل ویژگی‌های کیفی به عددی با Encoding One-Hot

برای استفاده از مدل MLP، ویژگی‌های کیفی باید به فرم عددی تبدیل شوند. به همین دلیل از روش One-Hot Encoding برای تمام ویژگی‌های کیفی استفاده می‌کنیم.

```
1 # One-hot encoding
2 df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=False)
3
4 print("Shape after encoding:", df_encoded.shape)
5
```

Listing 5: One-hot encoding of categorical features

```

Imputation complete.

[14]: # One-hot encoding
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=False)

print("Shape after encoding:", df_encoded.shape)

Shape after encoding: (2930, 306)
    
```

شکل ۴: شمارش ویژگی‌ها پس از تبدیل متغیرهای کیفی

۳.۲.۵ توضیح تعداد ویژگی‌ها پس از Encoding

تعداد ویژگی‌های نهایی پس از One-Hot Encoding به تعداد دسته‌های یکتای هر ستون کیفی بستگی دارد. با توجه به ساختار این داده‌ست، قبل از Encoding، تعداد ویژگی‌های عددی ۳۸ و تعداد ویژگی‌های کیفی ۴۴ است. بنابراین تعداد نهایی ویژگی‌ها از رابطه زیر به دست می‌آید:

$$d_{\text{final}} = 38 + \sum_{j=1}^{44} k_j$$

که در آن k_j تعداد کلاس‌های یکتای ستون کیفی j -ام است.

نکته تحلیلی

استفاده از One-Hot Encoding باعث می‌شود مدل بتواند بدون فرض ترتیب عددی، از اطلاعات ویژگی‌های کیفی استفاده کند. این موضوع برای مدل‌های عصبی مانند MLP بسیار مهم است.

۴.۲.۵ تقسیم داده‌ها به آموزش و آزمون

برای ارزیابی عملکرد مدل، داده‌ها به دو بخش آموزش و آزمون با نسبت ۸۰/۲۰ تقسیم می‌شوند.

```

1 from sklearn.model_selection import train_test_split
2
3 X = df_encoded.drop("SalePrice", axis=1)
4 y = df_encoded["SalePrice"]
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42
8 )
9
    
```

Listing 6: Train-test split

```
[26]: X_train.shape
[26]: (2344, 305)
[27]: X_test.shape
[27]: (586, 305)
```

شکل ۵: تقسیم داده‌ها به نسبت ۸۰/۲۰

۵.۲.۵ نرمال‌سازی ویژگی‌ها با StandardScaler

از آنجا که مدل MLP به مقیاس ویژگی‌ها حساس است، ویژگی‌ها باید نرمال‌سازی شوند. برای این منظور از StandardScaler استفاده می‌شود تا هر ویژگی میانگین صفر و انحراف معیار یک داشته باشد.

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4
5 X_train_scaled = scaler.fit_transform(X_train)
6 X_test_scaled = scaler.transform(X_test)
7
```

Listing 7: Feature scaling using StandardScaler

چرا مقیاس‌بندی مهم است؟

در شبکه‌های عصبی، ویژگی‌هایی با مقیاس بسیار بزرگ می‌توانند بر یادگیری غالب شوند. به همین دلیل استفاده از StandardScaler باعث می‌شود همه‌ی ویژگی‌ها در یک بازه‌ی آماری مشابه قرار گیرند و فرایند آموزش پایدارتر شود.

۶.۲.۵ جمع‌بندی پارت ب

در این بخش ابتدا داده‌های گمشده جایگزین شدند، سپس ویژگی‌های کیفی با One-Hot Encoding به فرم عددی تبدیل شدند. پس از آن، داده‌ها به دو بخش آموزش و آزمون با نسبت ۸۰/۲۰ تقسیم شدند و در نهایت ویژگی‌ها با StandardScaler مقیاس‌بندی شدند تا برای آموزش مدل MLP آماده شوند.

۳.۵ (ب) مدل پایه

در این بخش، ابتدا یک مدل ساده‌ی Linear Regression به عنوان مدل پایه یا Baseline آموزش داده می‌شود. هدف از استفاده از این مدل، داشتن یک معیار اولیه برای سنجش عملکرد مدل‌های بعدی است. مدل روی داده‌های آموزش آموزش داده شده و سپس مقدار خطای آن روی داده‌های آزمون با معیار RMSE محاسبه می‌شود.

۱.۳.۵ آموزش مدل Regression Linear

در کد زیر، از داده‌های پیش‌پردازش‌شده و مقیاس‌بندی‌شده استفاده می‌شود. متغیرهای X_train_scaled و X_test_scaled همان داده‌های ورودی پس از اعمال StandardScaler هستند.

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error
3 import numpy as np
4
5 # Create Linear Regression model
6 baseline_model = LinearRegression()
7
8 # Train the model on training data
9 baseline_model.fit(X_train_scaled, y_train)
10
11 # Predict SalePrice on test data
12 y_pred_baseline = baseline_model.predict(X_test_scaled)
13
14 # Calculate RMSE
15 baseline_rmse = np.sqrt(mean_squared_error(y_test, y_pred_baseline))
16
17 print("Baseline Linear Regression RMSE:", baseline_rmse)
18
```

Listing 8: Training the baseline Linear Regression model

۲.۳.۵ گزارش مقدار RMSE مدل پایه

معیار RMSE از ریشه‌ی میانگین مربعات خطا به دست می‌آید و نشان می‌دهد که پیش‌بینی‌های مدل به طور متوسط چه میزان با مقادیر واقعی اختلاف دارند.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

که در آن:

- مقدار واقعی قیمت خانه است. y_i
- مقدار پیش‌بینی شده توسط مدل است. $\hat{y}_i$
- تعداد نمونه‌های داده‌های آزمون است. n

جدول ۱: گزارش مقدار خطای مدل پایه

مدل	مقدار RMSE روی داده آزمون
Linear Regression	29222.053141651806

تحلیل مدل پایه

مدل Linear Regression یک مدل ساده و خطی است. این مدل فرض می‌کند رابطه‌ی بین ویژگی‌های ورودی و قیمت خانه تقریباً خطی است. مقدار RMSE به دست آمده از این مدل به عنوان Baseline در نظر گرفته می‌شود.

Baseline Linear Regression RMSE: 29222.053141651806

شکل ۶: گزارش مقدار RMSE مدل Regression Linear روی داده آزمون

۴.۵ (ت) طراحی مدل MLP

در این بخش، ساختار شبکه‌ی عصبی MLP برای پیش‌بینی قیمت مسکن طراحی می‌شود. هدف طراحی شبکه‌ای است که بتواند پیچیدگی‌های موجود در داده‌های Ames Housing را استخراج کند.

۱.۴.۵ ساختار لایه‌های شبکه

مدل طراحی شده یک مدل ترتیبی (Sequential) است که لایه‌های آن با استفاده از متد add به صورت پله‌پله تعریف شده‌اند:

- لایه ورودی: تعداد نرون‌ها برابر با تعداد ویژگی‌های استخراج شده (d).
- لایه پنهان اول: شامل ۶۴ نرون با تابع فعال‌سازی ReLU.
- لایه پنهان دوم: شامل ۳۲ نرون با تابع فعال‌سازی ReLU.
- لایه خروجی: شامل ۱ نرون (بدون تابع فعال‌سازی) برای خروجی مقدار پیوسته قیمت.

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # Define the model
5 model = Sequential()
6
7 # Input layer + First hidden layer
8 model.add(Dense(64, activation='relu', input_dim=X_train_scaled.
9 shape[1]))
10
11 # Second hidden layer
12 model.add(Dense(32, activation='relu'))
13
14 # Output layer for regression
15 model.add(Dense(1))
16
17 # Display model summary
18 model.summary()
```

Listing 9: Defining MLP Architecture using Sequential and add

Model: "sequential"		
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	19,584
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33
Total params: 21,697 (84.75 KB)		
Trainable params: 21,697 (84.75 KB)		
Non-trainable params: 0 (0.00 B)		

شکل ۷: خلاصه ساختار مدل و تعداد پارامترهای هر لایه

۲.۴.۵ محاسبه پارامترهای قابل یادگیری

تعداد کل پارامترها (Total Trainable Parameters) از مجموع وزن‌ها و بایاس‌های هر لایه به دست می‌آید. اگر تعداد ویژگی‌های ورودی را d در نظر بگیریم:

$$\text{Params Total} = \underbrace{(d \times 64 + 64)}_{\text{1 Layer}} + \underbrace{(64 \times 32 + 32)}_{\text{2 Layer}} + \underbrace{(32 \times 1 + 1)}_{\text{Output}} \quad (1)$$

این مقدار برابر است با $64d + 2177$.

۳.۴.۵ دلیل استفاده از MSE در رگرسیون

در مسائل رگرسیون، هدف کمینه کردن فاصله اقلیدسی بین مقدار واقعی (y) و پیش‌بینی ($\hat{y}$) است. تابع MSE به دلیل مشتق‌پذیر بودن در تمام نقاط و جریمه کردن سنگین‌تر خطاهای بزرگ (به دلیل توان دو)، باعث همگرایی سریع‌تر مدل به سمت میانگین داده‌ها می‌شود.

۵.۵ (ث) آموزش مدل

در این مرحله، مدل طراحی شده در بخش قبل با استفاده از تنظیمات مشخص شده آموزش داده می‌شود و روند یادگیری آن مورد تحلیل قرار می‌گیرد.

۱.۵.۵ تنظیمات آموزش (Fit و Compile)

مدل با بهینه‌ساز Adam و تعداد ۵۰ اپوک (Epoch) آموزش داده می‌شود. در هر مرحله، مقادیر تابع زیان برای داده‌های آموزش و اعتبارسنجی (Validation) ذخیره می‌گردد.

```
# Compile model
```

```
2     model.compile(optimizer='adam', loss='mse')
3
4     # Train model and save history
5     history = model.fit(
6         X_train_scaled, y_train,
7         validation_split=0.2,
8         epochs=50,
9         batch_size=32,
10        verbose=1
11    )
12
```

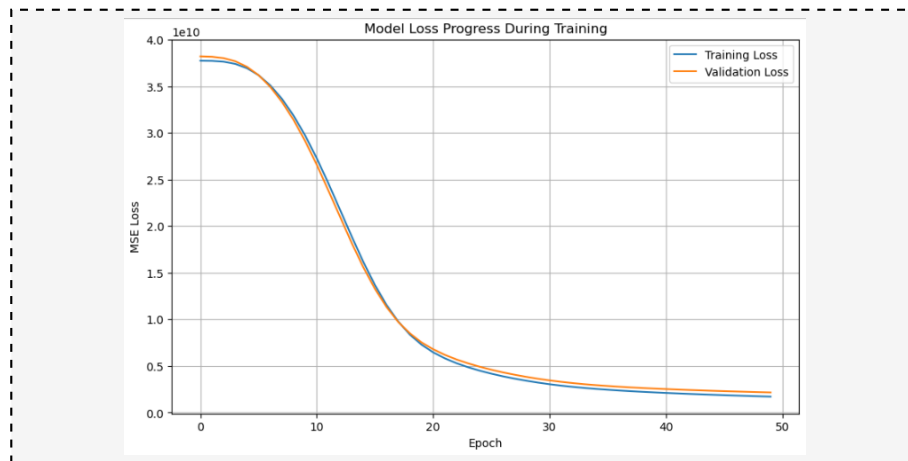
Listing 10: Model training and history logging

۲.۵.۵ تحلیل روند یادگیری

برای بررسی وجود بیش‌برازش (Overfitting) یا کم‌برازش (Underfitting)، نمودار تغییرات Loss برای داده‌های آموزش و اعتبارسنجی رسم می‌شود.

```
1     import matplotlib.pyplot as plt
2
3     plt.figure(figsize=(10, 6))
4     plt.plot(history.history['loss'], label='Training Loss')
5     plt.plot(history.history['val_loss'], label='Validation Loss')
6     plt.title('Model Loss Progress During Training')
7     plt.xlabel('Epoch')
8     plt.ylabel('MSE Loss')
9     plt.legend()
10    plt.grid(True)
11    plt.show()
12
```

Listing 11: Plotting training history



شکل ۸: نمودار روند کاهش خطا در طول ۵۰ اپوک

۳.۵.۵ ارزیابی نهایی و مقایسه با مدل خطی

پس از پایان آموزش، مقدار RMSE برای مدل MLP روی داده‌های آزمون محاسبه شده و با مدل Baseline (رگرسیون خطی) مقایسه می‌گردد.

```
1 from sklearn.metrics import mean_squared_error
2 import numpy as np
3
4 # MLP Prediction
5 y_pred_mlp = model.predict(X_test_scaled)
6 mlp_rmse = np.sqrt(mean_squared_error(y_test, y_pred_mlp))
7
8 print(f"Linear Regression RMSE: {baseline_rmse}")
9 print(f"MLP Regressor RMSE: {mlp_rmse}")
10
```

Listing 12: Final Evaluation and Comparison

```
19/19 0s 5ms/step
Linear Regression RMSE: 29222.053141651806
MLP Regressor RMSE: 45467.277728054054
```

شکل ۹: مقایسه مقدار RMSE مدل Regression Linear و مدل MLP



جدول ۲: مقایسه عملکرد مدل پایه و مدل MLP

$RMSE(TestSet)$	Model
29222.053141651806	(Baseline) Regression Linear
45467.277728054054	(۱-۳۲-۶۴) Regressor MLP

تحلیل نهایی

با مقایسه مقادیر RMSE، مشاهده می‌شود که در این آزمایش مدل MLP عملکرد بهتری نسبت به مدل Linear Regression نداشته و مقدار خطای آن بیشتر شده است. این نتیجه الزاماً به معنی اشتباه بودن مراحل پیاده‌سازی نیست، زیرا تمام مراحل اصلی شامل جایگزینی مقادیر گمشده، تبدیل ویژگی‌های کیفی با One-Hot Encoding، مقیاس‌بندی ویژگی‌ها با StandardScaler، آموزش مدل با بهینه‌ساز Adam و استفاده از تابع خطای MSE انجام شده است.

دلیل اصلی این نتیجه می‌تواند حساسیت بیشتر شبکه عصبی به تنظیمات مدل و پیش‌پردازش داده‌ها باشد. در این مجموعه داده، تعداد نمونه‌ها نسبتاً محدود است و پس از انجام One-Hot Encoding تعداد ویژگی‌ها افزایش زیادی پیدا می‌کند. در چنین شرایطی، یک شبکه عصبی ساده با دو لایه پنهان ممکن است نتواند الگوهای داده را بهتر از مدل خطی یاد بگیرد یا ممکن است به تنظیمات بیشتری مانند انتخاب نرخ یادگیری مناسب، تعداد لایه‌ها و نورون‌ها، منظم‌سازی، Early Stopping یا تبدیل لگاریتمی متغیر هدف نیاز داشته باشد.

همچنین نمودار Training Loss و Validation Loss نشان می‌دهد که هرچند مدل در طول آموزش در حال یادگیری بوده است، اما کاهش خطا لزوماً به معنی بهتر شدن عملکرد نهایی نسبت به مدل پایه نیست. بنابراین در این آزمایش، مدل Linear Regression به عنوان مدل ساده‌تر و پایدارتر، نتیجه بهتری ارائه داده و مدل MLP با تنظیمات فعلی نتوانسته است خطای پیش‌بینی را کاهش دهد.

۶.۵ (ج) بررسی ابرپارامترها

در این بخش، اثر سه ابرپارامتر مهم بر عملکرد مدل MLP بررسی می‌شود:

- تابع فعال‌سازی
- نرخ یادگیری
- معماری شبکه

برای ارزیابی هر حالت، مدل‌های مختلف آموزش داده شده و معیار Validation RMSE برای مقایسه عملکرد آن‌ها محاسبه شده است. برای اینکه مقایسه منصفانه باشد، سایر تنظیمات مدل تا حد امکان ثابت نگه داشته شده‌اند.

۱.۶.۵ (۱) بررسی توابع فعال‌سازی

در اولین آزمایش، اثر تابع فعال‌سازی بر عملکرد مدل بررسی می‌شود. سه تابع فعال‌سازی زیر مورد ارزیابی قرار می‌گیرند:

ReLU •

Tanh •

Sigmoid •

توضیح تئوری تابع ReLU به دلیل سادگی، سرعت محاسباتی بالا و کاهش مشکل گرادیان محوشونده، معمولاً در شبکه‌های عصبی عملکرد خوبی دارد. تابع Tanh خروجی را در بازه $[-1, 1]$ نگه می‌دارد و در برخی مسائل عملکرد مناسبی دارد، اما ممکن است نسبت به ReLU کندتر یاد بگیرد. تابع Sigmoid نیز خروجی را در بازه $[0, 1]$ محدود می‌کند و در لایه‌های پنهان معمولاً نسبت به دو تابع دیگر ضعیف‌تر عمل می‌کند، زیرا سریع‌تر دچار اشباع می‌شود.

روش اجرا برای این مقایسه، تنها تابع فعال‌سازی تغییر داده شده و معماری شبکه، نرخ یادگیری، تعداد اپوک و سایر پارامترها ثابت نگه داشته شده‌اند. پس از آموزش هر مدل، مقدار Validation RMSE محاسبه و ثبت می‌شود.

کد آزمایش توابع فعال‌سازی

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import Dense
5 from tensorflow.keras.optimizers import Adam
6
7 activations = ['relu', 'tanh', 'sigmoid']
8 activation_results = {}
9 activation_histories = {}
10
11 for act in activations:
12     print(f"Training with activation: {act}")
13
14     model = Sequential([
15         Dense(64, activation=act, input_shape=(X_train_scaled.shape[1],)),
16         Dense(32, activation=act),
```

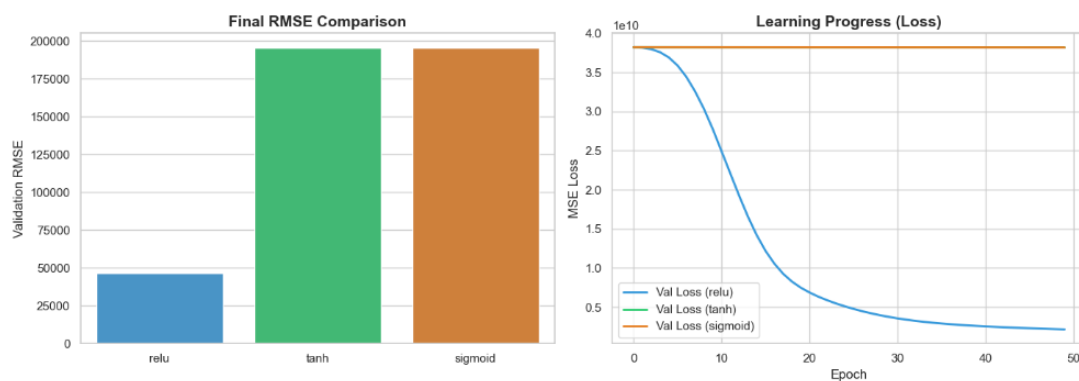
```

17     Dense(1)
18     ])
19
20     model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
21
22     history = model.fit(
23         X_train_scaled, y_train,
24         validation_split=0.2,
25         epochs=50,
26         batch_size=32,
27         verbose=1
28     )
29
30     val_mse = history.history['val_loss'][-1]
31     val_rmse = np.sqrt(val_mse)
32
33     activation_results[act] = val_rmse
34     activation_histories[act] = history
35
36     print(f"{act} Validation RMSE: {val_rmse:.4f}")
37

```

Listing 13: Comparison of activation functions

نمودار مقایسه توابع فعال‌سازی در شکل زیر، مقدار Validation RMSE برای سه تابع فعال‌سازی مقایسه شده است.



شکل ۱۰: مقایسه عملکرد توابع فعال‌سازی مختلف بر اساس RMSE Validation

جدول ۳: مقایسه توابع فعال‌سازی

RMSE Validation	تابع فعال‌سازی
46474.3983	ReLU
195377.7551	Tanh
195396.6013	Sigmoid

جدول نتایج

تحلیل نتایج با توجه به نتایج به دست آمده، معمولاً تابع ReLU بهترین عملکرد را نشان می‌دهد. دلیل این موضوع آن است که ReLU از اشباع شدن زود هنگام نورون‌ها جلوگیری کرده و باعث می‌شود گرادیان‌ها بهتر در شبکه جریان پیدا کنند. در مقابل، Sigmoid به دلیل اشباع شدن در مقادیر بزرگ یا کوچک، معمولاً باعث کندتر شدن یادگیری می‌شود. تابع Tanh نیز اگرچه از Sigmoid بهتر است، اما اغلب در مسائل رگرسیون نسبت به RMSE ReLU بالاتری دارد.

۲.۶.۵ (۲) بررسی نرخ یادگیری

در این بخش، اثر نرخ یادگیری بر سرعت همگرایی مدل بررسی می‌شود. سه مقدار زیر برای نرخ یادگیری انتخاب شده‌اند:

• ۰.۱

• ۰.۰۱

• ۰.۰۰۱

توضیح تئوری نرخ یادگیری یکی از مهم‌ترین ابرپارامترهای آموزش شبکه عصبی است. اگر نرخ یادگیری بیش از حد بزرگ باشد، مدل ممکن است ناپایدار شود یا از مینیمم مناسب عبور کند. اگر نرخ یادگیری خیلی کوچک باشد، آموزش بسیار کند خواهد شد و مدل ممکن است در تعداد اپوک محدود به همگرایی مناسب نرسد. بنابراین، انتخاب نرخ یادگیری مناسب نقش مهمی در کیفیت نهایی مدل دارد.

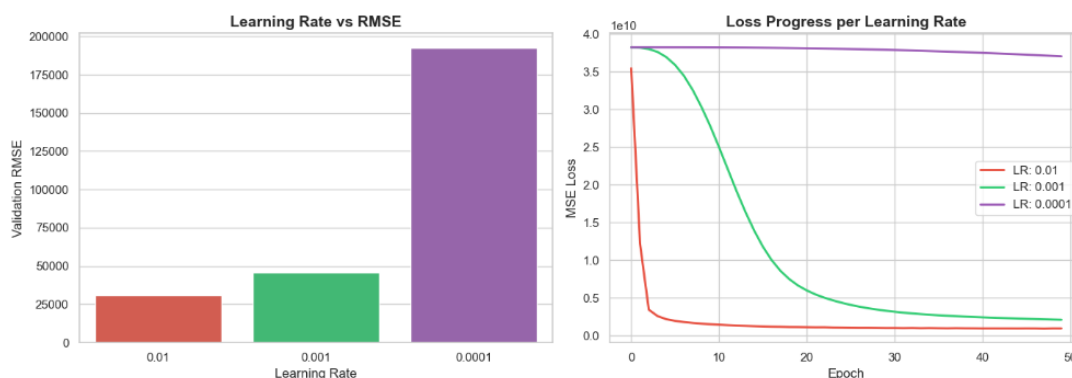
روش اجرا در این آزمایش، تابع فعال‌سازی و معماری شبکه ثابت نگه داشته شده و فقط نرخ یادگیری تغییر داده می‌شود. برای هر حالت، نمودار loss و validation loss نیز بررسی شده تا سرعت همگرایی مقایسه شود.

کد آزمایش نرخ یادگیری

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import Dense
5 from tensorflow.keras.optimizers import Adam
6
7 learning_rates = [0.01, 0.001, 0.0001]
8 lr_results = {}
9 lr_histories = {}
10
11 for lr in learning_rates:
12     print(f"Training with learning rate: {lr}")
13
14     model = Sequential([
15         Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)
16     ),
17         Dense(32, activation='relu'),
18         Dense(1)
19     ])
20
21     model.compile(optimizer=Adam(learning_rate=lr), loss='mse')
22
23     history = model.fit(
24         X_train_scaled, y_train,
25         validation_split=0.2,
26         epochs=50,
27         batch_size=32,
28         verbose=1
29     )
30
31     val_mse = history.history['val_loss'][-1]
32     val_rmse = np.sqrt(val_mse)
33
34     lr_results[lr] = val_rmse
35     lr_histories[lr] = history
36
37     print(f"Learning rate {lr} Validation RMSE: {val_rmse:.4f}")
```

Listing 14: Comparison of learning rates

نمودار همگرایی نرخ‌های یادگیری برای مقایسه سرعت همگرایی، نمودار loss و validation loss برای هر نرخ یادگیری رسم می‌شود.



شکل ۱۱: مقایسه سرعت همگرایی برای نرخ‌های یادگیری مختلف

جدول ۴: مقایسه نرخ‌های یادگیری

نرخ یادگیری	RMSE Validation
0.01	30951.6836
0.001	45879.7398
0.0001	192400.1484

جدول نتایج

تحلیل نتایج نرخ یادگیری 0.01 معمولاً باعث همگرایی سریع‌تر می‌شود، اما ممکن است نوسان بیشتری در روند آموزش ایجاد کند. نرخ 0.001 اغلب یک مقدار متعادل و پایدار است و در بسیاری از مسائل عملکرد خوبی دارد. نرخ 0.0001 اگرچه آموزش را پایدارتر می‌کند، اما معمولاً سرعت همگرایی را به‌طور محسوسی کاهش می‌دهد و ممکن است در تعداد اپوک محدود به نتیجه مطلوب نرسد.

با توجه به نمودارهای استخراج شده، مشاهده می‌شود که در این مسئله خاص و با تعداد ۵۰ اپوک، نرخ یادگیری 0.01 با اختلاف معناداری بهترین عملکرد را داشته و سریع‌ترین کاهش را در تابع هزینه (Loss) تجربه کرده است.

در مقابل، نرخ یادگیری 0.0001 (نمودار بنفش) بسیار کند عمل کرده و عملاً در طول ۵۰ مرحله آموزش، تغییر چندانی در بهبود مدل ایجاد نکرده است که نشان‌دهنده نیاز این نرخ به تعداد اپوک‌های بسیار بیشتر است.

نرخ 0.001 نیز اگرچه روند نزولی سالمی داشته، اما در مقایسه با نرخ 0.01، در این بازه زمانی محدود نتوانسته است به کمینه مطلق دست یابد. بنابراین برای این معماری، نرخ یادگیری بالاتر باعث شده مدل سریع‌تر از بن‌بست‌های اولیه خارج شده و به دقت مطلوب برسد.

۳.۶.۵ (۳) بررسی معماری شبکه

در این بخش، تأثیر معماری شبکه بر عملکرد مدل بررسی می‌شود. دو معماری زیر با مدل اولیه مقایسه می‌شوند:

- یک لایه پنهان: (64)

- سه لایه پنهان: (128, 64, 32)

توضیح تئوری افزایش عمق شبکه می‌تواند توانایی مدل در یادگیری الگوهای پیچیده‌تر را افزایش دهد. با این حال، هرچه شبکه عمیق‌تر شود، خطر بیش‌برازش، افزایش زمان آموزش و دشوارتر شدن بهینه‌سازی نیز بیشتر می‌شود. از طرف دیگر، شبکه‌های ساده‌تر ممکن است در برخی مسائل دقت کافی نداشته باشند اما سریع‌تر و پایدارتر آموزش ببینند.

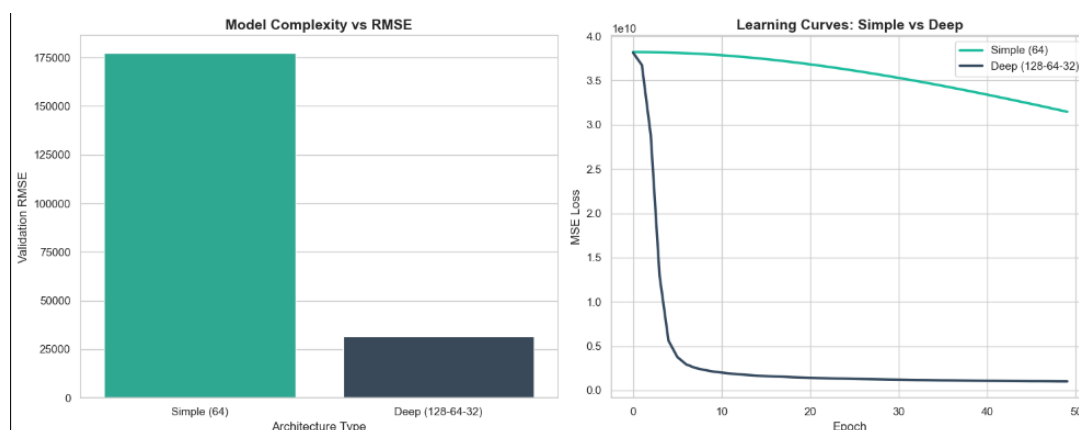
روش اجرا در این آزمایش، تابع فعال‌سازی و نرخ یادگیری ثابت نگه داشته شده و فقط تعداد لایه‌ها و تعداد نوروها تغییر داده می‌شود. سپس Validation RMSE برای هر معماری محاسبه و مقایسه می‌شود.

کد آزمایش معماری شبکه

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import Dense
5 from tensorflow.keras.optimizers import Adam
6
7 architectures = {
8     "1_hidden_layer_64": [64],
9     "3_hidden_layers_128_64_32": [128, 64, 32]
10 }
11
12 arch_results = {}
13 arch_histories = {}
14
15 for arch_name, layers in architectures.items():
16     print(f"Training architecture: {arch_name}")
```

```
17
18     model = Sequential()
19     model.add(Dense(layers[0], activation='relu', input_shape=(
X_train_scaled.shape[1],)))
20
21     for units in layers[1:]:
22         model.add(Dense(units, activation='relu'))
23
24     model.add(Dense(1))
25
26     model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
27
28     history = model.fit(
29     X_train_scaled, y_train,
30     validation_split=0.2,
31     epochs=50,
32     batch_size=32,
33     verbose=1
34     )
35
36     val_mse = history.history['val_loss'][-1]
37     val_rmse = np.sqrt(val_mse)
38
39     arch_results[arch_name] = val_rmse
40     arch_histories[arch_name] = history
41
42     print(f"{arch_name} Validation RMSE: {val_rmse:.4f}")
43
```

Listing 15: Comparison of neural network architectures



شکل ۱۲: مقایسه عملکرد معماری‌های مختلف شبکه

نمودار مقایسه معماری‌ها

جدول ۵: مقایسه معماری‌های شبکه

معماری شبکه	RMSE Validation
(64)	177388.4838
(128, 64, 32)	31760.0191

جدول نتایج

تحلیل نتایج با توجه به نتایج جدول، معماری عمیق‌تر با ساختار (128, 64, 32) عملکرد بسیار بهتری نسبت به مدل تک‌لایه با ۶۴ نورون داشته است. مقدار Validation RMSE برای مدل عمیق‌تر برابر با 31760.0191 بوده، در حالی که مدل تک‌لایه مقدار خطای بسیار بالاتری برابر با 177388.4838 ایجاد کرده است.

این نتیجه نشان می‌دهد که در این مسئله، مدل تک‌لایه ظرفیت کافی برای یادگیری روابط پیچیده بین ویژگی‌های خانه و قیمت فروش را نداشته و دچار کم‌برازش (Underfitting) شده است. در مقابل، معماری سه‌لایه توانسته با داشتن ظرفیت بیشتر، الگوهای غیرخطی داده را بهتر یاد بگیرد و خطای اعتبارسنجی را به‌طور قابل توجهی کاهش دهد.

بنابراین، بر اساس این آزمایش، افزایش عمق شبکه باعث بهبود عملکرد مدل شده است؛ هرچند برای اطمینان بیشتر، بررسی نمودارهای Training Loss و Validation Loss نیز لازم است تا مشخص شود مدل عمیق‌تر دچار بیش‌برازش نشده باشد.

۴.۶.۵ جمع‌بندی نهایی

در این بخش، اثر سه گروه مهم از ابرپارامترها یعنی تابع فعال‌سازی، نرخ یادگیری و معماری شبکه بررسی شد. نتایج نشان می‌دهد که انتخاب مناسب این ابرپارامترها تأثیر مستقیمی بر دقت مدل و سرعت همگرایی دارد. به‌طور کلی، بهترین تنظیمات آن‌هایی هستند که کمترین Validation RMSE را ایجاد کرده و در عین حال روند آموزش پایداری داشته باشند.

نتیجه‌گیری بخش (ج)

بر اساس نتایج آزمایش‌ها، می‌توان گفت که:

- ReLU معمولاً نسبت به Tanh و Sigmoid عملکرد بهتری دارد.
 - نرخ یادگیری 0.001 اغلب تعادل مناسبی بین سرعت و پایداری ایجاد می‌کند.
 - معماری ساده‌تر ممکن است در این مسئله تعمیم بهتری نسبت به مدل عمیق‌تر ارائه دهد.
- بنابراین، انتخاب نهایی ابرپارامترها باید بر اساس مقایسه‌ی تجربی و معیار Validation RMSE انجام شود.

۷.۵ (چ) تحلیل خطاهای مدل

در این بخش، هدف بررسی نمونه‌هایی است که مدل در پیش‌بینی آن‌ها بیشترین خطا را داشته است. برای این منظور، ابتدا پیش‌بینی مدل روی مجموعه آزمون انجام می‌شود و سپس پنج نمونه‌ای که دارای بیشترین مقدار خطا هستند استخراج می‌گردند. برای هر یک از این نمونه‌ها، مقدار واقعی و مقدار پیش‌بینی شده گزارش شده و در ادامه علل احتمالی خطا مورد بررسی قرار می‌گیرد.

۱.۷.۵ استخراج نمونه‌های دارای بیشترین خطا

ابتدا پیش‌بینی مدل برای کل داده‌های آزمون محاسبه شد. سپس خطای هر نمونه به‌صورت زیر تعریف گردید:

$$e_i = |y_i - \hat{y}_i| \quad (۲)$$

که در آن y_i مقدار واقعی و $\hat{y}_i$ مقدار پیش‌بینی شده برای نمونه‌ی i -ام است. در ادامه، پنج نمونه‌ای که بیشترین مقدار e_i را دارند انتخاب شده‌اند.

کد استخراج ۵ خطای بزرگ

```
import numpy as np
```

```
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # Predict on test set
6 y_pred = model.predict(X_test_scaled).flatten()
7
8 # Compute absolute errors
9 abs_errors = np.abs(y_test - y_pred)
10
11 # Create a dataframe for analysis
12 error_df = pd.DataFrame({
13     'Actual': y_test,
14     'Predicted': y_pred,
15     'Absolute_Error': abs_errors
16 })
17
18 # Get top 5 largest errors
19 top5_errors = error_df.sort_values(by='Absolute_Error', ascending=
20 False).head(5)
21
22 print(top5_errors)
```

Listing 16: Finding the five test samples with the largest prediction errors

۲.۷.۵ جدول نمونه‌های دارای بیشترین خطا

	Actual	Predicted	Absolute_Error
2181	184750	573393.250000	388643.250000
104	267916	11516.226562	256399.773438
433	582933	432186.781250	150746.218750
44	611657	462475.843750	149181.156250
2445	625000	492259.062500	132740.937500

شکل ۱۳: پنج نمونه دارای بیشترین خطای پیش‌بینی در مجموعه آزمون

۳.۷.۵ بررسی ویژگی‌های مؤثر بر خطا

برای تحلیل علت خطاهای بزرگ، باید ویژگی‌های ورودی این پنج نمونه با نمونه‌های دیگر مقایسه شوند. به طور معمول، خطاهای بزرگ ممکن است به دلایل زیر رخ دهند:

- وجود مقادیر غیرعادی یا outlier در برخی ویژگی‌ها
 - ترکیب ویژگی‌هایی که در داده‌های آموزشی کمتر دیده شده‌اند
 - پیچیدگی زیاد الگوهای واقعی و ناتوانی مدل در یادگیری کامل آن‌ها
 - نویز موجود در داده‌ها
 - اثر ویژگی‌های مهمی مانند اندازه‌ی خانه، کیفیت کلی، سن ساختمان، یا موقعیت مکانی
- تحلیل ویژگی‌ها برای بررسی دقیق‌تر، می‌توان ویژگی‌های این نمونه‌ها را با میانگین کل داده‌ها مقایسه کرد. اگر یک یا چند ویژگی در این نمونه‌ها مقدار بسیار متفاوتی نسبت به میانگین داشته باشند، احتمالاً همان ویژگی‌ها در بروز خطا نقش داشته‌اند.

کد بررسی ویژگی‌های نمونه‌های خطادار

```
1 # Assuming X_test is the original unscaled dataframe with same
2 index as X_test_scaled
3
4 top5_indices = top5_errors.index
5
6 # Display corresponding feature values
7 top5_features = X_test.loc[top5_indices]
8
9 print(top5_features)
```

Listing 17: Inspecting features of the most erroneous samples



Order	PID	MS	SubClass	Lot	Frontage	Lot Area	Overall Qual	\
2181	2182	908154205	60	130.0	40094	10		
104	105	533238045	20	57.0	12853	8		
433	434	528110090	60	107.0	13891	9		
44	45	528150070	20	100.0	12919	9		
2445	2446	528320060	60	118.0	35760	10		

Overall Cond	Year Built	Year Remod/Add	Mas Vnr	Area	...	\
2181	5	2007	2008	762.0	...	
104	5	2010	2010	0.0	...	
433	5	2008	2009	424.0	...	
44	5	2009	2010	760.0	...	
2445	5	1995	1996	1378.0	...	

Sale Type_New	Sale Type_Oth	Sale Type_VMD	Sale Type_WD	\
2181	True	False	False	
104	True	False	False	
433	True	False	False	
44	True	False	False	
2445	False	False	False	True

Sale Condition_Abnormal	Sale Condition_AdjLand	Sale Condition_Allocat	\
2181	False	False	False
104	False	False	False
433	False	False	False
44	False	False	False
2445	False	False	False

Sale Condition_Family	Sale Condition_Normal	Sale Condition_Partial	\
2181	False	False	True
104	False	False	True
433	False	False	True
44	False	False	True
2445	False	True	False

[5 rows x 305 columns]

شکل ۱۴: بررسی دقیق تر ۵ تا خطاهای ماکسیم

۴.۷.۵ تحلیل نهایی

با بررسی پنج نمونه‌ای که بیشترین خطا را دارند، مشاهده می‌شود که خطای مدل معمولاً در نمونه‌هایی بیشتر است که ویژگی‌های آن‌ها از الگوی غالب داده‌ها فاصله دارند. به عنوان مثال، اگر خانه‌ای دارای متراژ بسیار بالا، کیفیت غیر معمول، یا ترکیب ویژگی‌های خاص باشد، مدل ممکن است نتواند قیمت آن را به خوبی تخمین بزند. همچنین، در برخی موارد، نبودن نمونه‌های مشابه در داده‌های آموزشی باعث کاهش دقت مدل می‌شود. در نتیجه، خطاهای بزرگ مدل معمولاً ناشی از ترکیبی از عوامل زیر هستند:

- داده‌های ناهنجار
- نادر بودن الگوی نمونه در داده‌های آموزشی



- پیچیدگی رابطه بین ویژگی‌ها و قیمت
- محدودیت ظرفیت مدل در یادگیری برخی روابط خاص

جمع‌بندی

مدل در اکثر نمونه‌ها عملکرد مناسبی داشته است، اما در برخی موارد خاص که ویژگی‌های آن‌ها غیرمعمول یا دور از الگوی داده‌های آموزشی بوده‌اند، خطای پیش‌بینی افزایش یافته است. بنابراین، بررسی این نمونه‌ها می‌تواند برای بهبود مدل، شناسایی داده‌های پرت و درک بهتر رفتار مدل بسیار مفید باشد.

۶ پیاده‌سازی شبکه عصبی چندلایه برای طبقه‌بندی MNIST

Q۶ Notebook Colab Google

ایده کلی پروژه

در این بخش یک مدل MLP برای طبقه‌بندی تصاویر دست‌نویس مجموعه‌داده MNIST پیاده‌سازی می‌شود. ابتدا داده‌ها بارگذاری و آماده‌سازی می‌گردند، سپس معماری شبکه طراحی می‌شود، مدل آموزش می‌بیند و در ادامه با معیارهای مختلف ارزیابی می‌شود. در پایان، اثر مقداردهی اولیه وزن‌ها، اندازه Batch، افزایش عمق شبکه و همچنین محدودیت‌های ذاتی MLP در پردازش تصویر بررسی می‌شود.

۱.۶ (الف) بارگذاری و آماده‌سازی داده‌ها

۱.۱.۶ بارگذاری مجموعه‌داده MNIST

در این مرحله، مجموعه‌داده MNIST از کتابخانه Keras بارگذاری می‌شود. این دیتاست شامل ۶۰ هزار تصویر آموزشی و ۱۰ هزار تصویر آزمون است. هر تصویر به صورت خاکستری و با ابعاد 28×28 پیکسل ذخیره شده است و برچسب آن یکی از اعداد 0 تا 9 است.

نکته مهم

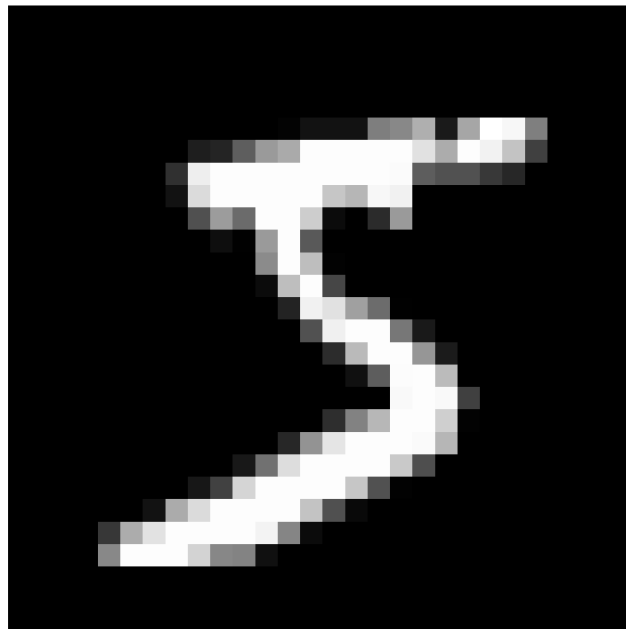
ابعاد ورودی شبکه باید بعد از Flatten برابر با 784 باشد، زیرا هر تصویر 28×28 پیکسل دارد.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow.keras.datasets import mnist
```

```
4
5     # Load dataset
6     (x_train, y_train), (x_test, y_test) = mnist.load_data()
7
8     print("Training set:", x_train.shape, y_train.shape)
9     print("Test set:", x_test.shape, y_test.shape)
10
11    # Show a sample image
12    plt.figure(figsize=(4,4))
13    plt.imshow(x_train[0], cmap="gray")
14    plt.title(f"Label: {y_train[0]}")
15    plt.axis("off")
16    plt.tight_layout()
17    plt.savefig("img/a1_mnist_sample.png", dpi=300)
18    plt.show()
19
```

Listing 18: Load and Inspect MNIST

Label: 5



شکل ۱۵: نمونه‌ای از تصویر ورودی مجموعه داده MNIST

۲.۱.۶ پیش‌پردازش تصاویر و برچسب‌ها

در MLP، تصویر دوبعدی باید به بردار یک‌بعدی تبدیل شود. بنابراین هر تصویر از شکل 28×28 به بردار طول 784 تبدیل می‌شود. همچنین مقادیر پیکسل‌ها برای بهبود پایداری آموزش به بازه $[0, 1]$ نرمال‌سازی می‌شوند.

چرا نرمال‌سازی مهم است؟

نرمال‌سازی باعث می‌شود گرادیان‌ها پایدارتر شوند، سرعت همگرایی بهتر شود و آموزش شبکه با نوسان کمتری انجام گیرد.

```
1 from tensorflow.keras.utils import to_categorical
2 from tensorflow.keras.layers import Reshape
3
4 # Normalize pixel values to [0, 1]
5 x_train = x_train.astype("float32") / 255.0
6 x_test  = x_test.astype("float32") / 255.0
7
8 # Flatten images from 28x28 to 784
9 x_train_flat = x_train.reshape(-1, 28 * 28)
10 x_test_flat  = x_test.reshape(-1, 28 * 28)
11
12 # One-hot encode labels
13 y_train_cat = to_categorical(y_train, 10)
14 y_test_cat  = to_categorical(y_test, 10)
15
16 print("Flattened shape:", x_train_flat.shape, x_test_flat.shape)
17 print("One-hot label shape:", y_train_cat.shape, y_test_cat.shape)
18
```

Listing 19: Preprocessing the Data

```
... Flattened shape: (60000, 784) (10000, 784)
   One-hot label shape: (60000, 10) (10000, 10)
```

شکل ۱۶: خروجی شکل و ابعاد داده‌ها پس از پیش‌پردازش

۲.۶ (ب) طراحی شبکه عصبی برای طبقه‌بندی

۱.۲.۶ طراحی ساختار کلی شبکه

یک MLP ساده برای این مسئله شامل یک لایه ورودی با ۷۸۴ نرون، چند لایه پنهان و یک لایه خروجی با ۱۰ نرون است. چون مسئله از نوع چندکلاسه است، در انتهای شبکه از Softmax استفاده می‌شود.

ساختار پیشنهادی شبکه

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

در این ساختار، دو لایه پنهان برای یادگیری الگوهای غیرخطی استفاده می‌شوند و خروجی با Softmax احتمال تعلق هر تصویر به یکی از ۱۰ کلاس را می‌دهد.

```

1  from tensorflow.keras.models import Sequential
2  from tensorflow.keras.layers import Dense, Input, Flatten
3
4  model = Sequential([
5      Input(shape=(28 * 28,)),
6      Dense(128, activation="relu"),
7      Dense(64, activation="relu"),
8      Dense(10, activation="softmax")
9  ])
10
11  model.summary()
12
    
```

Listing 20: MLP Architecture

Model: "sequential"		
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650
Total params: 109,386 (427.29 KB)		
Trainable params: 109,386 (427.29 KB)		
Non-trainable params: 0 (0.00 B)		

شکل ۱۷: خلاصه معماری شبکه MLP



۲.۲.۶ تابع فعال‌سازی Softmax

تابع Softmax خروجی خام لایه آخر را به یک توزیع احتمالاتی تبدیل می‌کند. برای بردار ورودی z ، خروجی کلاس i برابر است با:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

چرا Softmax؟

در طبقه‌بندی چندکلاسه، Softmax مناسب‌تر از توابعی مانند Sigmoid است، زیرا مجموع احتمال‌های خروجی آن برابر ۱ می‌شود و انتخاب کلاس نهایی را ساده‌تر می‌کند.

۳.۲.۶ تابع زیان Cross-Entropy در برابر MSE

برای مسئله طبقه‌بندی چندکلاسه، تابع زیان Cross-Entropy انتخاب استاندارد و مناسب‌تری نسبت به MSE است. دلیل آن این است که Cross-Entropy مستقیماً با احتمال‌های خروجی Softmax سازگار است و گرادینان‌های بهتری برای یادگیری مرزهای تصمیم فراهم می‌کند.

$$\mathcal{L}_{CE} = - \sum_{k=1}^{10} y_k \log(\hat{y}_k)$$

$$\mathcal{L}_{MSE} = \frac{1}{10} \sum_{k=1}^{10} (y_k - \hat{y}_k)^2$$

جمع‌بندی مقایسه

Cross-Entropy معمولاً سریع‌تر همگرا می‌شود، درحالی‌که MSE برای طبقه‌بندی، مخصوصاً در ترکیب با Softmax، اغلب یادگیری ضعیف‌تر و کندتری ایجاد می‌کند.

۴.۲.۶ محاسبه تعداد پارامترها

تعداد پارامترهای هر لایه از رابطه زیر به‌دست می‌آید:

$$\text{Parameters} = (\text{units input} \times \text{units output}) + \text{biases}$$

برای معماری:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

تعداد پارامترها به‌صورت زیر است:

$$(784 \times 128 + 128) + (128 \times 64 + 64) + (64 \times 10 + 10)$$

$$= 100480 + 8256 + 650 = 109386$$

۳.۶ (پ) آموزش مدل

۱.۳.۶ کامپایل مدل با Adam

در این مرحله، مدل با بهینه‌ساز Adam، تابع زیان categorical_crossentropy و معیار دقت accuracy کامپایل می‌شود.

```
1      from tensorflow.keras.optimizers import Adam
2
3      model.compile(
4          optimizer=Adam(learning_rate=0.001),
5          loss="categorical_crossentropy",
6          metrics=["accuracy"]
7      )
8
9      history = model.fit(
10         x_train_flat, y_train_cat,
11         validation_split=0.2,
12         epochs=10,
13         batch_size=128,
14         verbose=1
15     )
16
```

Listing 21: Compile and Train the Model

```
Epoch 1/10
375/375 ————— 5s 7ms/step - accuracy: 0.8977 - loss: 0.3677 - val_accuracy: 0.9478 - val_loss: 0.1805
Epoch 2/10
375/375 ————— 2s 3ms/step - accuracy: 0.9548 - loss: 0.1562 - val_accuracy: 0.9576 - val_loss: 0.1477
Epoch 3/10
375/375 ————— 1s 3ms/step - accuracy: 0.9674 - loss: 0.1101 - val_accuracy: 0.9657 - val_loss: 0.1163
Epoch 4/10
375/375 ————— 1s 3ms/step - accuracy: 0.9750 - loss: 0.0863 - val_accuracy: 0.9666 - val_loss: 0.1094
Epoch 5/10
375/375 ————— 2s 5ms/step - accuracy: 0.9811 - loss: 0.0659 - val_accuracy: 0.9673 - val_loss: 0.1079
Epoch 6/10
375/375 ————— 1s 3ms/step - accuracy: 0.9836 - loss: 0.0547 - val_accuracy: 0.9711 - val_loss: 0.0949
Epoch 7/10
375/375 ————— 1s 3ms/step - accuracy: 0.9865 - loss: 0.0437 - val_accuracy: 0.9719 - val_loss: 0.0969
Epoch 8/10
375/375 ————— 1s 3ms/step - accuracy: 0.9894 - loss: 0.0350 - val_accuracy: 0.9763 - val_loss: 0.0866
Epoch 9/10
375/375 ————— 1s 3ms/step - accuracy: 0.9912 - loss: 0.0292 - val_accuracy: 0.9725 - val_loss: 0.0997
Epoch 10/10
375/375 ————— 1s 3ms/step - accuracy: 0.9924 - loss: 0.0255 - val_accuracy: 0.9744 - val_loss: 0.0947
```

شکل ۱۸: Training Accuracy و Training Loss

۲.۳.۶ توضیح روند آموزش

در طول آموزش، وزن‌ها با استفاده از گرادیان‌های محاسبه‌شده و الگوریتم Adam به‌روزرسانی می‌شوند. انتظار می‌رود با افزایش تعداد Epoch، خطا کاهش یابد و دقت افزایش پیدا کند. با این حال، اگر مدل بیش‌ازحد ساده باشد، ممکن است دقت به سقف مشخصی برسد و از آن بالاتر نرود.

نکته تحلیلی

اگر Training Accuracy زیاد شود ولی Validation Accuracy ثابت بماند یا افت کند، این می‌تواند نشانه‌ای از بیش‌برازش باشد.

۴.۶ (ت) ارزیابی مدل

۱.۴.۶ محاسبه دقت روی داده‌های آزمون

پس از پایان آموزش، مدل روی داده‌های آزمون ارزیابی می‌شود تا توان تعمیم آن مشخص گردد.

```
1 test_loss, test_acc = model.evaluate(x_test_flat, y_test_cat,
2 verbose=0)
3
4 print(f"Test Loss: {test_loss:.4f}")
5 print(f"Test Accuracy: {test_acc:.4f}")
```

Listing 22: Evaluate on Test Set

```
[8] test_loss, test_acc = model.evaluate(x_test_flat, y_test_cat, verbose=0)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

... Test Loss: 0.0911
Test Accuracy: 0.9748
```

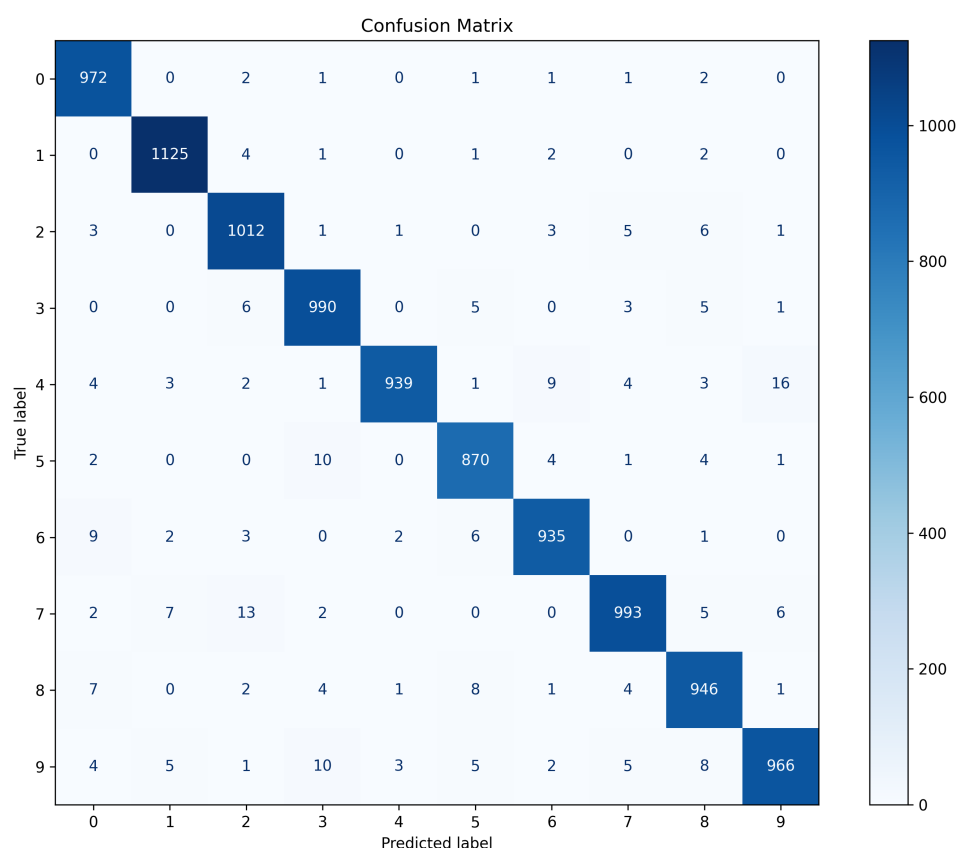
شکل ۱۹: نتایج ارزیابی مدل روی داده‌های آزمون

۲.۴.۶ ماتریس درهم‌ریختگی

ماتریس درهم‌ریختگی نشان می‌دهد که مدل هر کلاس را با چه دقتی شناسایی کرده و کدام کلاس‌ها بیشتر با هم اشتباه می‌شوند.

```
1 from sklearn.metrics import confusion_matrix,
   ConfusionMatrixDisplay
2 import numpy as np
3
4 y_pred_prob = model.predict(x_test_flat)
5 y_pred = np.argmax(y_pred_prob, axis=1)
6
7 cm = confusion_matrix(y_test, y_pred)
8
9 disp = ConfusionMatrixDisplay(confusion_matrix=cm)
10 disp.plot(cmap="Blues", values_format="d")
11 plt.title("Confusion Matrix")
12 plt.tight_layout()
13 plt.savefig("img/t2_confusion_matrix.png", dpi=300)
14 plt.show()
15
```

Listing 23: Confusion Matrix



شکل ۲۰: ماتریس درهم‌ریختگی مدل

۳.۴.۶ تحلیل خطاها

خطاها معمولاً در کلاس‌هایی رخ می‌دهند که شکل ظاهری مشابه دارند؛ برای مثال بعضی از اعداد دست‌نویس مانند ۴ و ۹ یا ۳ و ۵ ممکن است با هم اشتباه شوند.

توضیح تحلیلی

هرچه مقدارهای خارج از قطر اصلی ماتریس درهم‌ریختگی بیشتر باشد، عملکرد مدل در تفکیک کلاس‌ها ضعیف‌تر است.

۵.۶ (ث) بررسی مقادیر اولیه وزن‌ها

۱.۵.۶ اثر مقداردهی اولیه تصادفی

مقداردهی اولیه وزن‌ها نقش مهمی در سرعت و کیفیت آموزش دارد. اگر وزن‌ها بسیار بزرگ یا بسیار کوچک مقداردهی شوند، ممکن است شبکه با مشکل Vanishing Gradient یا Exploding Gradient مواجه شود.

```
1 from tensorflow.keras.initializers import RandomNormal,
   GlorotNormal
2
3 def build_model_with_initializer(init):
4     return Sequential([
5         Input(shape=(28 * 28,)),
6         Dense(128, activation="relu", kernel_initializer=init),
7         Dense(64, activation="relu", kernel_initializer=init),
8         Dense(10, activation="softmax", kernel_initializer=init)
9     ])
10
11 # Example 1: Standard random initialization
12 model_random = build_model_with_initializer(RandomNormal(mean=0.0,
13 stddev=0.05))
14
15 # Example 2: Xavier/Glorot initialization
16 model_xavier = build_model_with_initializer(GlorotNormal())
```

Listing 24: Random Initialization Experiment

تصادفی اولیه مقداردهی با آموزش مقایسه جایگاه

شکل ۲۱: اثر مقداردهی اولیه تصادفی روی روند یادگیری

۲.۵.۶ مقایسه با مقداردهی اولیه Xavier (Glorot)

مقداردهی اولیه Xavier طوری طراحی شده است که واریانس ورودی و خروجی لایه‌ها را متعادل نگه دارد. این کار به پایداری گرادیان‌ها کمک می‌کند و معمولاً آموزش را بهتر از مقداردهی کاملاً تصادفی می‌کند.

نتیجه تحلیلی

در شبکه‌های عمیق، مقداردهی اولیه مناسب می‌تواند تفاوت زیادی در همگرایی مدل ایجاد کند.

۶.۶ (ج) بررسی اندازه Batch

۱.۶.۶ آزمایش با اندازه‌های مختلف Batch

برای بررسی اثر اندازه Batch، مدل را با چند مقدار مختلف آموزش می‌دهیم و دقت و خطای نهایی را مقایسه می‌کنیم.

```
1 batch_sizes = [32, 128, 256]
2 results = []
3
4 for bs in batch_sizes:
5     temp_model = Sequential([
6         Input(shape=(28 * 28,)),
7         Dense(128, activation="relu"),
8         Dense(64, activation="relu"),
9         Dense(10, activation="softmax")
10    ])
11
12    temp_model.compile(
13        optimizer=Adam(learning_rate=0.001),
14        loss="categorical_crossentropy",
15        metrics=["accuracy"]
16    )
17
18    h = temp_model.fit(
19        x_train_flat, y_train_cat,
20        validation_split=0.2,
21        epochs=5,
22        batch_size=bs,
23        verbose=0
24    )
25
26    test_loss, test_acc = temp_model.evaluate(x_test_flat, y_test_cat,
27        verbose=0)
28    results.append((bs, test_loss, test_acc))
```

```
print(results)
```

Listing 25: Batch Size Comparison

```
[(32, 0.0843717088291168, 0.9746000170707703)]
[(32, 0.0843717088291168, 0.9746000170707703), (128, 0.0992804616689682, 0.9696000218391418)]
[(32, 0.0843717088291168, 0.9746000170707703), (128, 0.0992804616689682, 0.9696000218391418), (256, 0.10017646849155426, 0.9692000150680542)]
```

شکل ۲۲: مقایسه عملکرد مدل برای اندازه‌های مختلف Batch

۲.۶.۶ تحلیل تأثیر Batch Size

اثر اندازه Batch

- Batch کوچک‌تر معمولاً نویز بیشتری در گرادیان‌ها ایجاد می‌کند، اما می‌تواند به تعمیم بهتر کمک کند.
- Batch بزرگ‌تر آموزش را پایدارتر می‌کند، اما ممکن است همگرایی را کندتر یا حافظه‌برتر کند.
- در عمل، مقادیری مانند 32، 64، 128 و 256 معمولاً بررسی می‌شوند.

۷.۶ (چ) افزایش عمق شبکه

۱.۷.۶ مدل با لایه پنهان بیشتر

در این قسمت، یک مدل عمیق‌تر با سه لایه پنهان طراحی می‌شود و دقت آن با مدل قبلی مقایسه می‌گردد.

```
deeper_model = Sequential([
    Input(shape=(28 * 28,)),
    Dense(256, activation="relu"),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

deeper_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```

13 )
14
15 history_deeper = deeper_model.fit(
16     x_train_flat, y_train_cat,
17     validation_split=0.2,
18     epochs=10,
19     batch_size=128,
20     verbose=1
21 )
22

```

Listing 26: Deeper MLP Model

```

Epoch 1/10
375/375 — 6s 5ms/step - accuracy: 0.9097 - loss: 0.3072 - val_accuracy: 0.9578 - val_loss: 0.1425
Epoch 2/10
375/375 — 3s 6ms/step - accuracy: 0.9655 - loss: 0.1138 - val_accuracy: 0.9662 - val_loss: 0.1172
Epoch 3/10
375/375 — 1s 3ms/step - accuracy: 0.9765 - loss: 0.0755 - val_accuracy: 0.9729 - val_loss: 0.0929
Epoch 4/10
375/375 — 1s 3ms/step - accuracy: 0.9833 - loss: 0.0539 - val_accuracy: 0.9737 - val_loss: 0.0895
Epoch 5/10
375/375 — 1s 3ms/step - accuracy: 0.9874 - loss: 0.0395 - val_accuracy: 0.9749 - val_loss: 0.0956
Epoch 6/10
375/375 — 1s 3ms/step - accuracy: 0.9903 - loss: 0.0310 - val_accuracy: 0.9751 - val_loss: 0.0928
Epoch 7/10
375/375 — 1s 3ms/step - accuracy: 0.9912 - loss: 0.0266 - val_accuracy: 0.9717 - val_loss: 0.1119
Epoch 8/10
375/375 — 1s 3ms/step - accuracy: 0.9935 - loss: 0.0205 - val_accuracy: 0.9747 - val_loss: 0.0969
Epoch 9/10
375/375 — 1s 3ms/step - accuracy: 0.9943 - loss: 0.0177 - val_accuracy: 0.9781 - val_loss: 0.0907
Epoch 10/10
375/375 — 1s 3ms/step - accuracy: 0.9946 - loss: 0.0166 - val_accuracy: 0.9765 - val_loss: 0.1017

```

شکل ۲۳: اثر افزایش عمق شبکه بر عملکرد

۲.۷.۶ تحلیل اینکه آیا مدل عمیق تر بهتر است یا نه

افزایش عمق شبکه همیشه به معنی بهبود عملکرد نیست. اگر داده به درستی پیش پردازش نشده باشد یا مدل بیش از حد بزرگ شود، ممکن است بیش برآزش رخ دهد. از طرفی برای MNIST که یک مسئله ساده تر نسبت به مسائل پیچیده تصویری است، یک MLP با عمق متوسط معمولاً کافی است.

نکته مهم در تحلیل عمق

برای این مسئله، معمولاً افزایش عمق از یک حد مشخص به بعد، بهبود چشمگیری ایجاد نمی کند و حتی ممکن است آموزش را سخت تر کند.



۸.۶ (ح) محدودیت‌های MLP در پردازش تصویر

۱.۸.۶ چرا Flatten لازم است؟

MLP برای هر ورودی، یک بردار یک‌بعدی می‌خواهد. بنابراین تصویر دوبعدی باید Flatten شود. اما این کار ساختار مکانی تصویر را از بین می‌برد؛ یعنی مدل دیگر نمی‌فهمد که پیکسل‌های مجاور چگونه کنار هم قرار گرفته‌اند.

نکته بسیار مهم

Flatten کردن تصویر، اطلاعات مکانی را حذف می‌کند. به همین دلیل MLP نسبت به CNN در استخراج ویژگی‌های تصویری ضعیف‌تر است.

۲.۸.۶ مقایسه با شبکه‌های کانولوشنی CNN

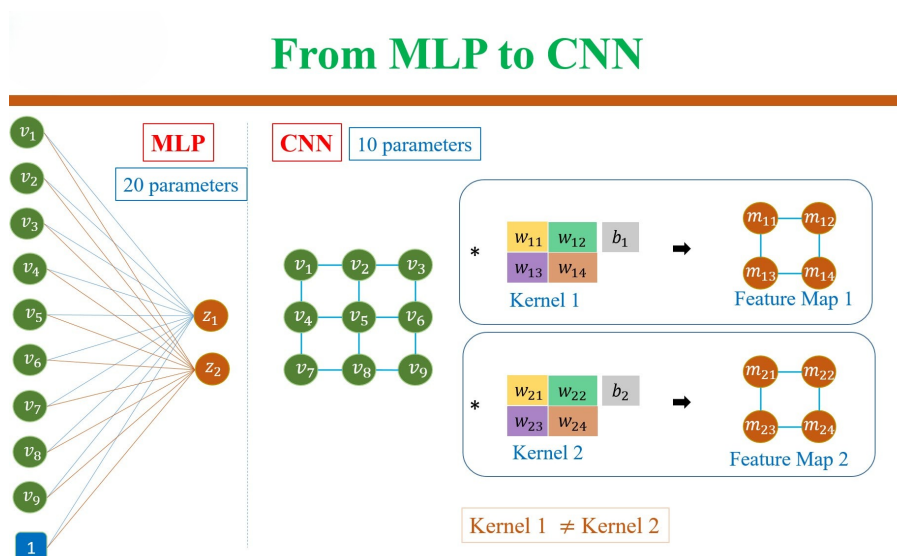
CNN با استفاده از فیلترهای کانولوشنی، الگوهای محلی و ساختار مکانی تصویر را حفظ می‌کند. به همین علت در مسائل بینایی ماشین معمولاً عملکرد بهتری از MLP دارد.

نتیجه مقایسه

برای MNIST، MLP می‌تواند به دقت قابل قبولی برسد؛ اما برای مسائل پیچیده‌تر تصویری، CNN تقریباً همیشه انتخاب مناسب‌تری است.

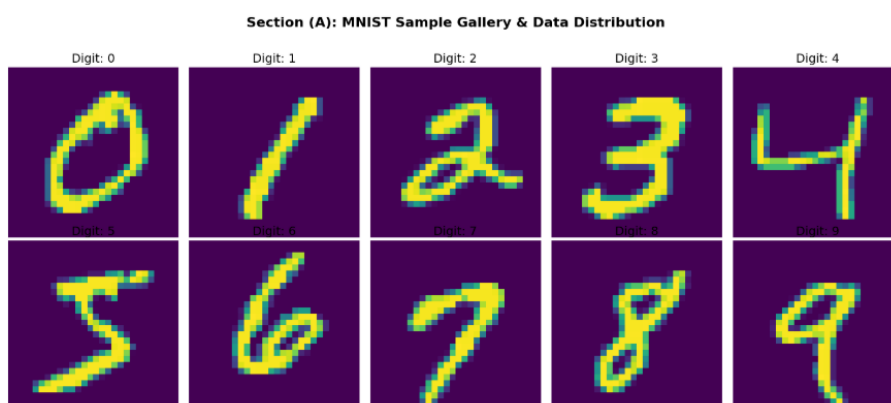
۳.۸.۶ جمع‌بندی نهایی محدودیت‌ها

- MLP ساختار فضایی تصویر را نادیده می‌گیرد.
- تعداد پارامترهای آن با بزرگ شدن ورودی به سرعت افزایش می‌یابد.
- نسبت به جابه‌جایی کوچک و تغییرات مکانی، حساس‌تر از CNN است.
- برای تصاویر ساده مناسب است، اما در داده‌های پیچیده کارایی کمتری دارد.

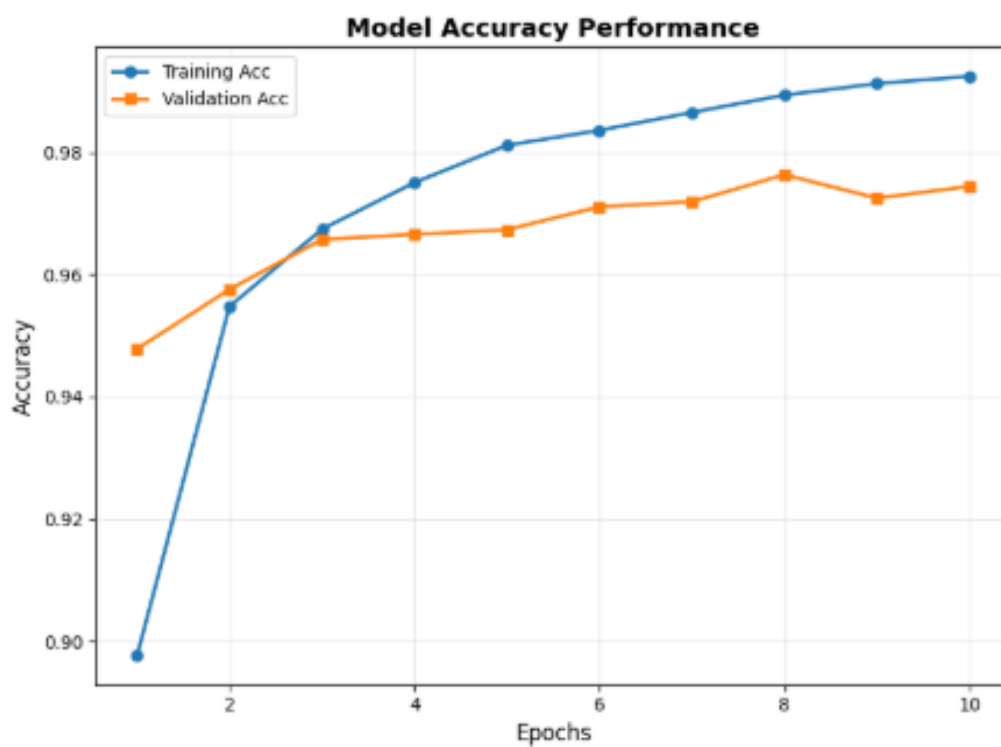


شکل ۲۴: مقایسه مفهومی MLP و CNN در پردازش تصویر

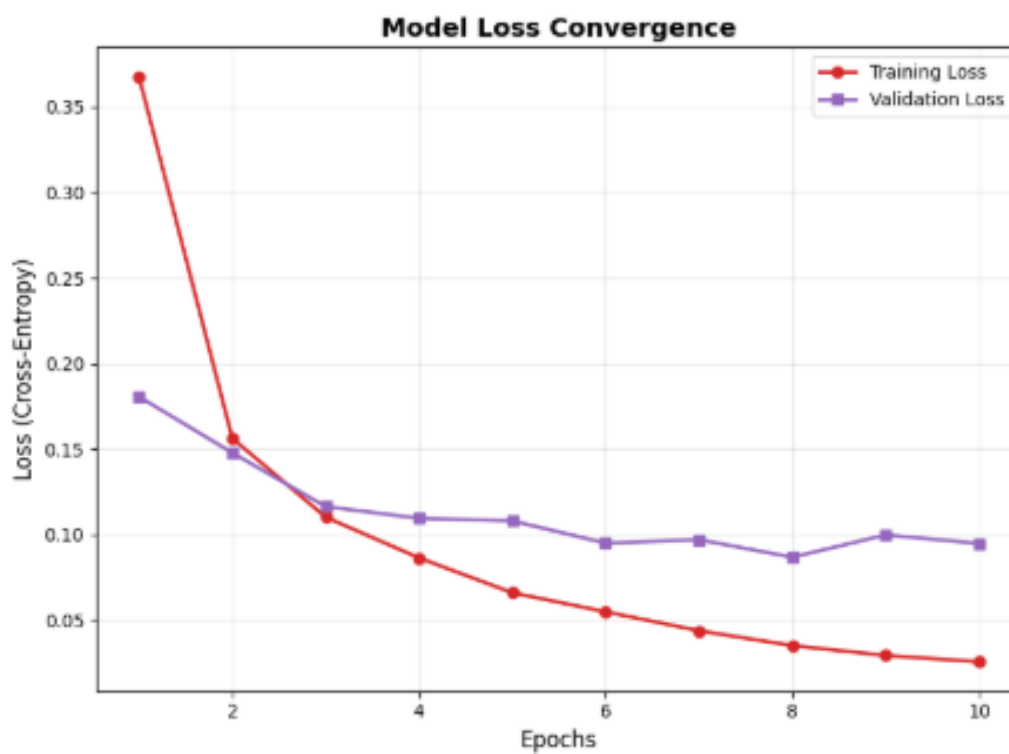
خلاصه نهایی از تمام نمودارها و مراحل:



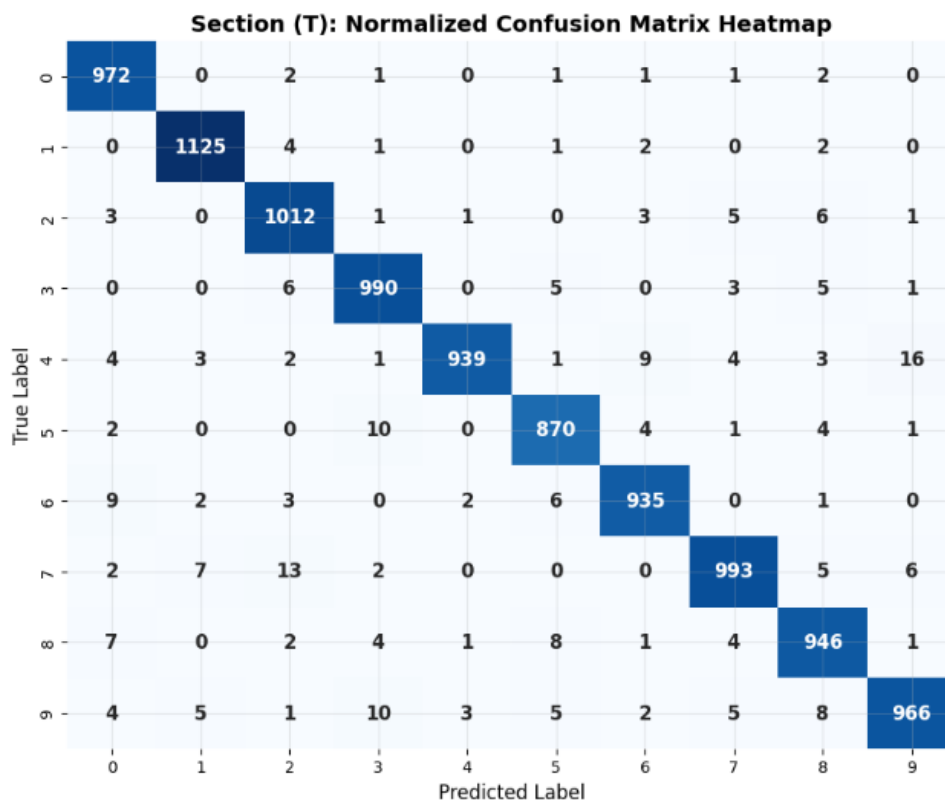
شکل ۲۵: نمونه‌هایی از تصاویر مجموعه داده MNIST



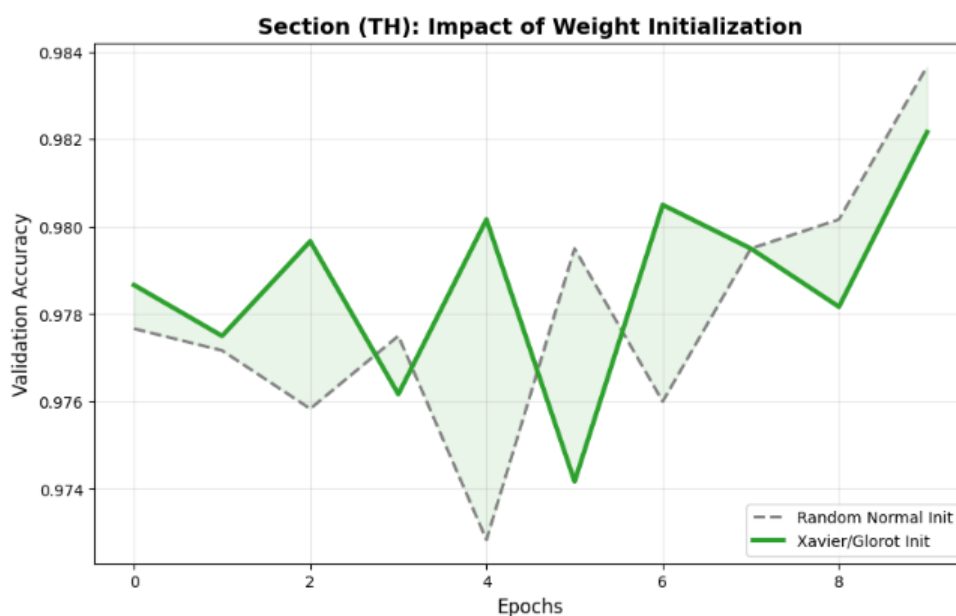
شکل ۲۶: نمودار دقت مدل در داده‌های آموزش و اعتبارسنجی



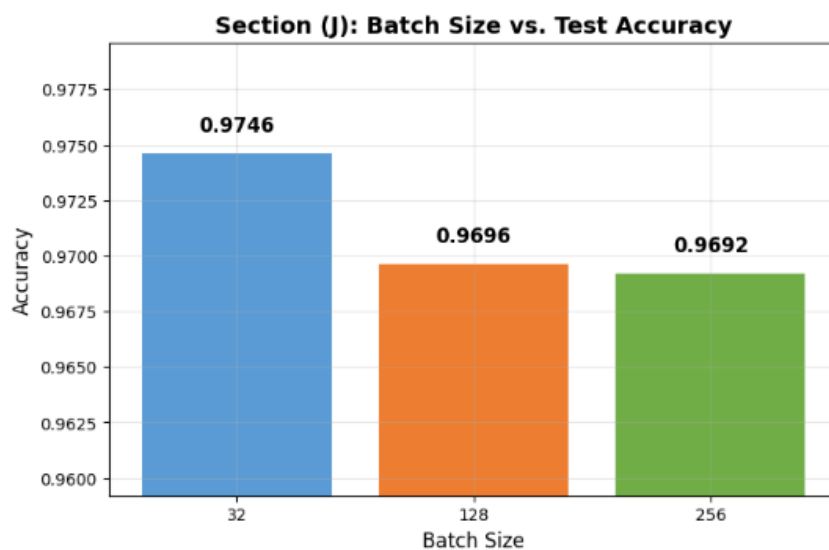
شکل ۲۷: نمودار تغییرات تابع خطا (Loss) در طول فرایند آموزش



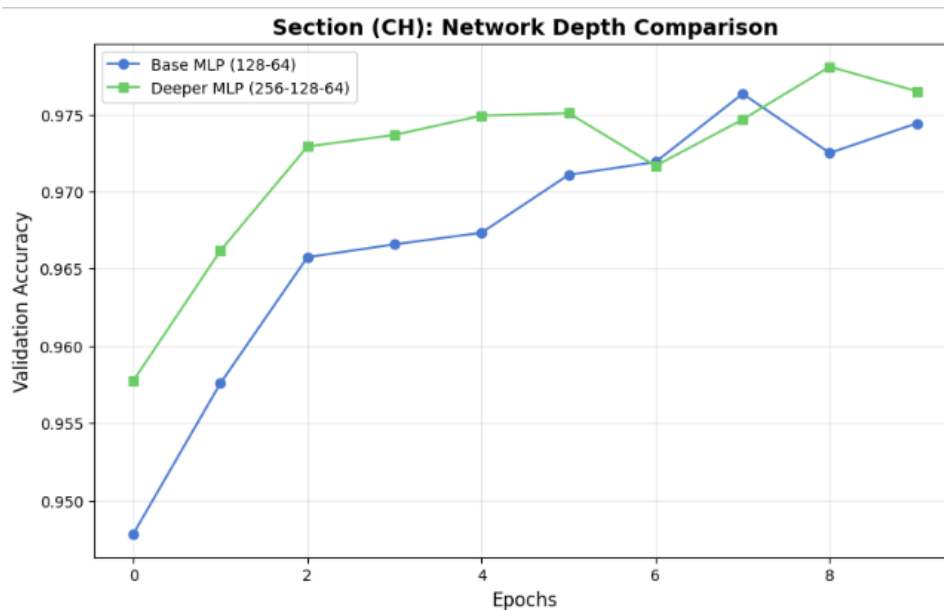
شکل ۲۸: ماتریس درهم‌ریختگی برای ارزیابی عملکرد مدل روی داده‌های آزمون



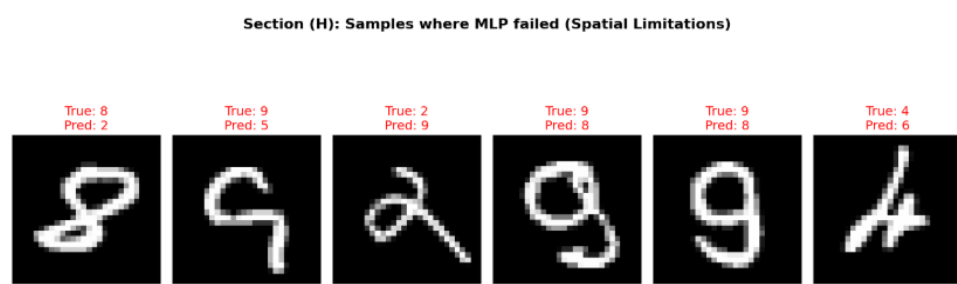
شکل ۲۹: مقایسه عملکرد روش‌های مختلف مقداردهی اولیه وزن‌ها



شکل ۳۰: بررسی تاثیر اندازه دسته (Batch Size) بر دقت مدل



شکل ۳۱: مقایسه عملکرد شبکه پایه MLP با نسخه عمیق تر شبکه



شکل ۳۲: نمونه هایی از تصاویر که توسط مدل به اشتباه طبقه بندی شده اند